TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA: CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN CUỐI KÌ MÔN NGÔN NGỮ LẬP TRÌNH TIÊN TIẾN

XÂY DỰNG WEBSITE MẠNG XÃ HỘI OKAYJI

Giảng viên hướng dẫn: **NGUYỄN MINH ĐẠO**

Mã môn học: **ADPL331379_01**

Tp. Hồ Chí Minh, tháng 04 năm 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA: CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN CUỐI KÌ MÔN NGÔN NGỮ LẬP TRÌNH TIÊN TIẾN

XÂY DỰNG WEBSITE MẠNG XÃ HỘI OKAYJI

Giảng viên hướng dẫn: **NGUYỄN MINH ĐẠO**

Mã môn học: **ADPL331379_01**

Sinh viên thực hiện: **Võ Lê Khánh Duy** **23110196**

Trần Triều Dương **23110200**

Tp. Hồ Chí Minh, tháng 04 năm 2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

Hình thức: .....

.....

.....

Nội dung: .....

.....

.....

Tổng điểm: .....

Ngày tháng 04 năm 2026

Giảng viên:

Nguyễn Minh Đạo

LỜI CẢM ƠN

Tập thể nhóm xin gửi cảm ơn chân thành đến thầy Nguyễn Minh Đạo – Giảng viên khoa công nghệ thông tin trường Đại học Công Nghệ Kỹ Thuật TP.HCM đã nhiệt tình và chu đáo trong việc hướng dẫn nhóm thực hiện đồ án.

Những chia sẻ của thầy là kinh nghiệm quý báu để chúng em áp dụng vào đề tài, cũng chính là hành trang để chúng em áp dụng vào công việc thực tế sau này.

Một lần nữa nhóm chúng em xin gửi lời cảm ơn đến thầy!

Tp. HCM, ngày 05 tháng 04 năm 2026

MỤC LỤC

PHẦN MỞ ĐẦU 1
1. Giới thiệu đề tài 1
2. Mục tiêu đề tài 1
PHẦN NỘI DUNG 3
CHƯƠNG 1: PHÂN TÍCH YÊU CẦU 3
1.1. Yêu cầu chức năng 3
1.2. Yêu cầu phi chức năng 5
CHƯƠNG 2: THIẾT KẾ HỆ THỐNG 7
2.1. Mô tả các tập thực thể 7
2.2. Mô tả các mối quan hệ 8
2.3. Entity-Relationship Diagram (ERD) 12
2.4. Use Case Diagram 13
2.5. Đặc tả 2 chức năng chính 14
2.6. Activity Diagram 19
2.7. Sequence Diagram 21
2.8. Class Diagram 23
2.9. Package Diagram 25
CHƯƠNG 3: KIẾN TRÚC HỆ THỐNG 26
3.1. Tổng quan 26
3.2. Kiến trúc Back-end 27
3.3. Kiến trúc Front-end 29
CHƯƠNG 4: MÔ TẢ SOLID ĐÃ ÁP DỤNG 31
4.1. Single Responsibility Principle (SRP) 31
4.2. Open/Closed Principle (OCP) 31
4.3. Liskov Substitution Principle (LSP) 32
4.4. Interface Segregation Principle (ISP) 32
4.5. Dependency Inversion Principle (DIP) 33
CHƯƠNG 5: GIAO DIỆN VÀ KIỂM THỬ 34
5.1. Giao diện xác thực và quản lý hồ sơ người dùng 34
5.2. Giao diện bảng tin, bài viết và tương tác xã hội 36
5.3. Giao diện nhắn tin và thông báo 39

5.4. Giao diện Admin 40
5.5. Kiểm thử hệ thống 43
CHƯƠNG 6: KHÓ KHĂN VÀ HƯỚNG MỞ RỘNG 46
6.1. Khó khăn trong phân tích và thiết kế hệ thống 46
6.2. Khó khăn trong triển khai kỹ thuật 46
6.3. Hướng mở rộng của hệ thống 47
PHẦN KẾT LUẬN 49
TÀI LIỆU THAM KHẢO 50

PHẦN MỞ ĐẦU

1. Giới thiệu đề tài

Trong bối cảnh các nền tảng số ngày càng phát triển, nhu cầu xây dựng một hệ thống mạng xã hội hiện đại, hỗ trợ kết nối và tương tác trực tuyến giữa người dùng đang trở nên rất phổ biến. Đề tài "Xây dựng website mạng xã hội OkayJi" được thực hiện nhằm phát triển một ứng dụng cho phép người dùng đăng ký tài khoản, quản lý hồ sơ cá nhân, đăng bài viết, tương tác thông qua lượt thích và bình luận, kết bạn, nhắn tin trực tiếp và nhận thông báo theo thời gian thực.

Điểm nổi bật của hệ thống OkayJi là sự kết hợp giữa kiến trúc web hiện đại và các công nghệ hỗ trợ thời gian thực, giúp nâng cao trải nghiệm người dùng. Hệ thống được xây dựng với back-end sử dụng Java Spring Boot để cung cấp REST API, xác thực JWT, WebSocket, quản lý dữ liệu và tích hợp kiểm duyệt nội dung tự động; đồng thời front-end sử dụng ReactJS và Vite để xây dựng giao diện trực quan, linh hoạt và dễ sử dụng.

Bên cạnh các chức năng cơ bản của một mạng xã hội, đề tài còn hướng tới việc tạo ra một môi trường tương tác lành mạnh và an toàn thông qua cơ chế kiểm duyệt nội dung bằng AI. Khi người dùng đăng bài viết hoặc bình luận, hệ thống có thể tự động đánh giá nội dung, từ đó hỗ trợ phát hiện nội dung không phù hợp trước khi hiển thị công khai trên nền tảng.

Đề tài không chỉ mang ý nghĩa xây dựng một sản phẩm phần mềm hoàn chỉnh theo mô hình client-server, mà còn là cơ hội để áp dụng các kiến thức đã học về phân tích yêu cầu, thiết kế hệ thống, lập trình hướng đối tượng, thiết kế RESTful API, bảo mật ứng dụng web, xử lý real-time và tổ chức mã nguồn theo nguyên tắc SOLID.

2. Mục tiêu đề tài

Mục tiêu chính của đề tài là xây dựng một hệ thống mạng xã hội hoạt động ổn định, có đầy đủ các chức năng cốt lõi phục vụ nhu cầu giao tiếp và chia sẻ thông tin giữa người dùng. Hệ thống cần hỗ trợ các chức năng như đăng ký, đăng nhập, quản lý hồ sơ cá nhân, đăng bài viết, bình luận, thích bài viết, kết bạn, nhắn tin cá nhân hoặc theo nhóm, cũng như gửi thông báo theo thời gian thực.

Ngoài các chức năng nghiệp vụ, đề tài còn hướng đến mục tiêu áp dụng kiến trúc phần mềm hợp lý để hệ thống dễ phát triển, dễ bảo trì và dễ mở rộng trong tương lai. Phần back-end được tổ chức theo các lớp như controller, service và repository, kết hợp với cơ sở dữ liệu MySQL; phần front-end được tổ chức theo mô hình pages, components và context để quản lý giao diện và trạng thái ứng dụng một cách rõ ràng.

Một mục tiêu quan trọng khác là đảm bảo yếu tố bảo mật và an toàn nội dung cho hệ thống. Đề tài áp dụng cơ chế xác thực và phân quyền bằng JWT cùng Spring Security, đồng thời tích hợp AI moderation để hỗ trợ kiểm duyệt bài viết và bình luận, góp phần giảm thiểu nội dung độc hại hoặc không phù hợp trong môi trường mạng xã hội.

Bên cạnh đó, đề tài còn nhằm rèn luyện kỹ năng làm việc nhóm, phân chia công việc, quản lý mã nguồn và triển khai một dự án phần mềm tương đối hoàn chỉnh từ giai đoạn phân tích, thiết kế cho đến cài đặt và kiểm thử. Thông qua quá trình thực hiện OkayJi, nhóm có thể củng cố kiến thức chuyên môn và tích lũy kinh nghiệm thực tế trong việc xây dựng một hệ thống web hiện đại có khả năng tích hợp nhiều công nghệ khác nhau.

PHẦN NỘI DUNG

CHƯƠNG 1: PHÂN TÍCH YÊU CẦU

1.1. Yêu cầu chức năng

Hệ thống phải hỗ trợ đầy đủ các chức năng cốt lõi của một nền tảng mạng xã hội hiện đại, bao gồm quản lý tài khoản, quản lý hồ sơ cá nhân, đăng tải nội dung, tương tác cộng đồng, kết bạn, nhắn tin, thông báo thời gian thực và kiểm duyệt nội dung.

Quản lý tài khoản và xác thực người dùng:

Hệ thống phải cho phép người dùng đăng ký tài khoản mới, đăng nhập và đăng xuất khỏi hệ thống. Khi đăng nhập thành công, người dùng được cấp mã xác thực để sử dụng các chức năng yêu cầu quyền truy cập. Hệ thống cũng cần hỗ trợ cơ chế ghi nhớ đăng nhập trong một khoảng thời gian nhất định nhằm tăng tính tiện lợi khi sử dụng.

Ngoài ra, hệ thống phải hỗ trợ phân quyền người dùng theo vai trò, chẳng hạn như USER và ADMIN. Người dùng thông thường được sử dụng các chức năng cơ bản của mạng xã hội, trong khi quản trị viên có quyền quản lý, giám sát và xử lý các nội dung hoặc tài khoản vi phạm.

Quản lý bài viết:

Hệ thống phải cho phép người dùng tạo bài viết mới với nội dung văn bản, hình ảnh hoặc video. Người dùng có thể chỉnh sửa hoặc xóa bài viết của chính mình khi cần thiết. Ngoài ra, bài viết cần có các trạng thái xử lý như đang chờ kiểm duyệt, đã xuất bản, bị từ chối hoặc đang xem xét. Cơ chế này giúp hệ thống kiểm soát nội dung trước khi hiển thị công khai đến cộng đồng.

Tương tác với bài viết:

Người dùng phải có thể bày tỏ sự quan tâm đối với bài viết thông qua chức năng thích bài viết. Hệ thống cũng cần hỗ trợ tạo mới, chỉnh sửa và xóa bình luận để người dùng có thể trao đổi và thảo luận dưới mỗi bài đăng.

Quản lý quan hệ bạn bè:

Hệ thống phải hỗ trợ người dùng gửi lời mời kết bạn, chấp nhận hoặc từ chối lời mời đã nhận. Người dùng cũng cần có khả năng hủy kết bạn khi không còn nhu cầu duy trì kết nối.

Ngoài ra, hệ thống cần cho phép xem danh sách bạn bè hiện tại và danh sách lời mời đang chờ xử lý. Chức năng này là nền tảng để mở rộng quyền riêng tư, cá nhân hóa bảng tin và hỗ trợ giao tiếp giữa các người dùng có kết nối với nhau.

Nhắn tin và trò chuyện thời gian thực:

Hệ thống phải cho phép người dùng nhắn tin riêng giữa hai người và nhắn tin nhóm giữa nhiều thành viên. Người dùng cần có thể tạo nhóm chat, đặt tên nhóm, thêm hoặc xóa thành viên trong nhóm theo quyền hạn phù hợp.

Các tin nhắn phải được gửi và nhận theo thời gian thực để đảm bảo trải nghiệm giao tiếp tự nhiên. Hệ thống cũng cần hỗ trợ tải lịch sử tin nhắn theo từng đợt và quản lý nhiều loại nội dung tin nhắn như văn bản, hình ảnh và tệp đính kèm.

Thông báo thời gian thực:

Hệ thống phải cung cấp chức năng thông báo cho người dùng khi có các sự kiện quan trọng như có lượt thích, bình luận mới, lời mời kết bạn hoặc lời mời kết bạn được chấp nhận. Các thông báo này cần được hiển thị gần như ngay lập tức để người dùng không bỏ lỡ các tương tác liên quan đến mình.

Ngoài ra, hệ thống cũng cần hỗ trợ gửi thông báo từ phía hệ thống, ví dụ thông báo chung hoặc thông báo liên quan đến trạng thái tài khoản và nội dung.

Kiểm duyệt nội dung tự động:

Một chức năng quan trọng của hệ thống là kiểm duyệt nội dung bài viết và bình luận bằng AI. Khi người dùng gửi nội dung mới, hệ thống cần tạo tác vụ kiểm duyệt để đánh giá mức độ phù hợp của nội dung trước khi xuất bản.

Nếu nội dung đạt yêu cầu, bài viết hoặc bình luận có thể được công khai trên hệ thống. Ngược lại, nếu phát hiện dấu hiệu vi phạm, hệ thống cần từ chối nội dung hoặc chuyển sang trạng thái chờ xem xét. Chức năng này góp phần xây dựng môi trường mạng xã hội an toàn, văn minh và lành mạnh.

Quản lý tệp:

Hệ thống phải cho phép tải lên các tệp đa phương tiện như ảnh, video và một số tệp đính kèm khác trong giới hạn kích thước cho phép. Các tệp sau khi tải lên cần được lưu trữ an toàn và có thể truy cập thông qua liên kết tạm thời.

Quản lý báo cáo vi phạm

Hệ thống phải cho phép người dùng báo cáo nội dung vi phạm liên quan đến bài viết, bình luận, người dùng. Quản trị viên sẽ là người xem xét các báo cáo và đưa ra quyết định chấp nhận hay bác bỏ báo cáo. Tương ứng với quyết định chấp nhận báo cáo cần thực hiện các hành động là ẩn bài đăng vi phạm, xóa bình luận vi phạm hay khóa tài khoản người dùng vi phạm.

Quản lý kiểm duyệt thủ công

Hệ thống phải cho phép quản trị viên thay đổi trạng thái bài đăng hoặc xem xét các bài đăng có khả năng vi phạm nhưng chưa được hệ thống kiểm duyệt đánh giá chính xác.

Thống kê vận hành hệ thống

Hệ thống phải cho phép quản trị viên xem các báo cáo thống kê về tổng số người dùng, tổng số bài đăng, tổng số cuộc trò chuyện, số người dùng mới theo năm nhằm đánh giá tính hiệu quả của hệ thống khi đi vào hoạt động.

Tìm kiếm người dùng, bài viết

Hệ thống phải cho phép người dùng tìm kiếm bạn bè, bài viết mà họ quan tâm nhưng phải tuân thủ quyền riêng tư của người dùng khác. Chẳng hạn, một người không thể xem các bài viết của một người có chế độ hiển thị hồ sơ là "chỉ bạn bè" khi họ chưa kết bạn với người này.

1.2. Yêu cầu phi chức năng

Hiệu năng:

Hệ thống cần có thời gian phản hồi nhanh đối với các thao tác phổ biến như đăng nhập, tải bảng tin, xem hồ sơ, gửi bình luận và mở danh sách bạn bè. Đối với các chức năng thời gian thực như chat và thông báo, dữ liệu phải được cập nhật gần như ngay lập tức để bảo đảm trải nghiệm người dùng.

Ngoài ra, hệ thống cần hỗ trợ phân trang ở các màn hình có nhiều dữ liệu như bảng tin, danh sách bài viết, danh sách tin nhắn và thông báo. Việc này giúp giảm tải cho máy chủ và cải thiện tốc độ hiển thị trên giao diện người dùng.

Bảo mật:

Hệ thống phải bảo đảm chỉ người dùng đã xác thực mới được truy cập vào các chức năng yêu cầu đăng nhập. Các dữ liệu nhạy cảm như thông tin đăng nhập, mã xác thực và khóa cấu hình cần được bảo vệ an toàn, không lưu trữ hoặc hiển thị công khai không cần thiết.

Bên cạnh đó, hệ thống cần kiểm tra dữ liệu đầu vào để hạn chế các nguy cơ như gửi dữ liệu sai định dạng, truy cập trái phép hoặc khai thác lỗ hổng bảo mật.

Khả năng mở rộng, bảo trì:

Hệ thống cần được thiết kế theo hướng dễ mở rộng thêm chức năng trong tương lai. Mã nguồn của hệ thống cần được tổ chức rõ ràng, dễ đọc, dễ kiểm tra và dễ chỉnh sửa. Các lớp và module phải được phân chia theo đúng trách nhiệm để giảm phụ thuộc lẫn nhau và hỗ trợ bảo trì lâu dài. Hệ thống cũng nên tuân thủ các nguyên tắc thiết kế phần mềm như SOLID để tăng khả năng tái sử dụng và mở rộng.

Giao diện dễ sử dụng:

Giao diện người dùng cần được thiết kế trực quan, dễ thao tác và thống nhất giữa các màn hình chức năng. Người dùng mới phải có thể nhanh chóng làm quen với các thao tác cơ bản như đăng nhập, đăng bài, nhắn tin, kết bạn và chỉnh sửa hồ sơ.

CHƯƠNG 2: THIẾT KẾ HỆ THỐNG

2.1. Mô tả các tập thực thể

Hệ thống cơ sở dữ liệu được xây dựng từ nhiều tập thực thể chính gồm User, Profile, Role, UserRoles, InvalidatedToken, Post, PostMedia, Comment, Reaction, Friend, FriendRequest, Chat, ChatMember, Message, ModerationJob, ModerationResult và Notification; các thực thể này phối hợp với nhau để đáp ứng các chức năng quản lý tài khoản, đăng bài, tương tác xã hội, nhắn tin, kiểm duyệt nội dung và gửi thông báo trong hệ thống.

Nhóm người dùng:

User là tập thực thể trung tâm dùng để lưu thông tin tài khoản người dùng, bao gồm các thuộc tính như *id, username, email, password, status, created_at, updated_at, last_change_username, oauth_provider* và *oauth_id*, trong đó *id* là khóa chính.

Profile là tập thực thể mô tả hồ sơ cá nhân của người dùng, gồm các thuộc tính *user_id, full_name, gender, bio, birthday, avatar_url, cover_image_url* và *visibility*

InvalidatedToken là tập thực thể dùng để lưu các token đã bị vô hiệu hóa nhằm phục vụ chức năng bảo mật và xác thực, với các thuộc tính chính là *id* và *expiryTime*

Nhóm phân quyền và quan hệ xã hội:

Role là tập thực thể dùng để quản lý vai trò của người dùng trong hệ thống, gồm thuộc tính *name* làm khóa chính và *description* để mô tả vai trò, với các giá trị vai trò như *ADMIN* và *USER*.

UserRoles là tập thực thể kết hợp nhằm biểu diễn quan hệ nhiều-nhiều giữa User và Role, bao gồm hai thuộc tính *user_id* và *roles_name*, trong đó cả hai cùng tham gia khóa chính và đồng thời là khóa ngoại tham chiếu đến các bảng liên quan.

Friend là tập thực thể dùng để quản lý thông tin quan hệ bạn bè giữa hai người dùng thông qua các thuộc tính *id, user_low_id* và *user_high_id*

FriendRequest là tập thực thể dùng để quản lý yêu cầu kết bạn với các thuộc tính *id, created_at, receiver_id* và *sender_id*

Nhóm nội dung và tương tác:

Post là tập thực thể lưu bài viết do người dùng tạo ra, gồm các thuộc tính *id,*

user_id, content, created_at và *status*, trong đó mỗi bài viết thuộc về một người dùng.

PostMedia là tập thực thể lưu các tệp media gắn với bài viết, gồm *id, media_url, type* và *post_id*

Comment là tập thực thể lưu bình luận với các thuộc tính *id, post_id, user_id, content* và *created_at*.

Reaction là tập thực thể ghi nhận hành vi tương tác của người dùng đối với bài viết thông qua các thuộc tính *id, post_id, user_id* và *created_at*

Nhóm nhắn tin, kiểm duyệt và thông báo:

Chat là tập thực thể biểu diễn cuộc trò chuyện trong hệ thống, bao gồm các thuộc tính *id, chat_avatar_url, chat_name, type, created_by, direct_key, created_at, last_message_at* và *last_message_seq*, trong đó kiểu chat có thể là *DIRECT* hoặc *GROUP*.

ChatMember là tập thực thể kết hợp dùng để quản lý thành viên tham gia từng cuộc trò chuyện với các thuộc tính *id, chat_id, member_id, joined_at* và *last_read_seq*

Message lưu nội dung tin nhắn qua các thuộc tính *id, chat_id, type, sender_id, seq, content* và *created_at*

ModerationJob và **ModerationResult** lần lượt dùng để quản lý tiến trình kiểm duyệt và kết quả kiểm duyệt nội dung

Notification là tập thực thể lưu các thông báo gửi đến người dùng với các thuộc tính là *id, user_id, payload, type, is_read* và *created_at*

Report là tập thực thể lưu các báo cáo vi phạm liên quan đến bài đăng và bình luận với các thuộc tính *id, reporter_id, target_id, target_type, reason, details, status, created_at, resolved_by* và *resolved_at*

2.2. Mô tả các mối quan hệ

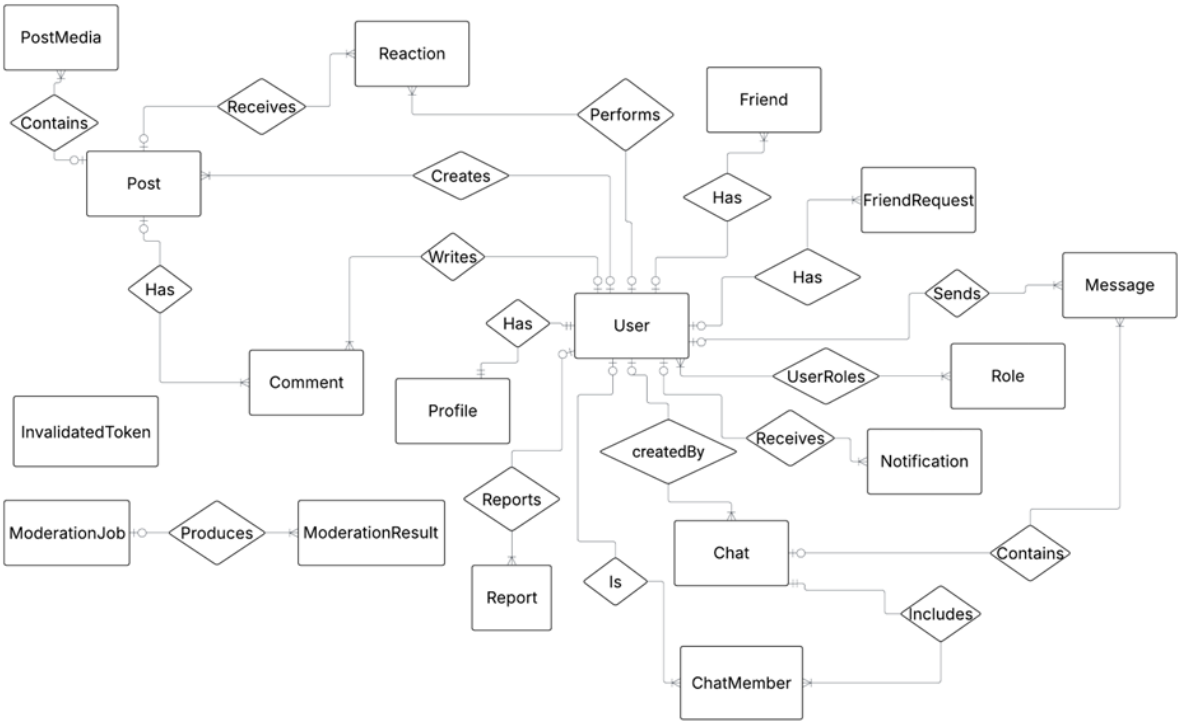
Nhóm	Thực thể	Liên kết với	Mô tả
Người dùng	User	Profile	Quan hệ 1-1; mỗi người dùng có một hồ sơ cá nhân và mỗi hồ sơ chỉ thuộc về một người dùng

Phân quyền	User	Role (qua UserRoles)	Quan hệ n-n; một người dùng có thể có nhiều vai trò và một vai trò có thể được gán cho nhiều người dùng
Quan hệ xã hội	User	Friend	Quan hệ 1-n; một người dùng có thể kết bạn với nhiều người dùng khác
Quan hệ xã hội	User	FriendRequest	Quan hệ 1-n; một người dùng có thể gửi hoặc nhận nhiều lời mời kết bạn
Nội dung	User	Post	Quan hệ 1-n; một người dùng có thể tạo nhiều bài viết, nhưng mỗi bài viết chỉ thuộc về một người dùng
Nội dung	Post	PostMedia	Quan hệ 1-n; một bài viết có thể chứa nhiều tệp phương tiện, còn mỗi bản ghi media chỉ gắn với một bài viết
Nội dung	Post	Comment	Quan hệ 1-n; một bài viết có thể có nhiều bình luận, còn mỗi bình luận chỉ thuộc về một bài viết
Nội dung	User	Comment	Quan hệ 1-n; một người dùng có thể viết nhiều bình luận, nhưng mỗi bình luận chỉ do một người dùng tạo ra

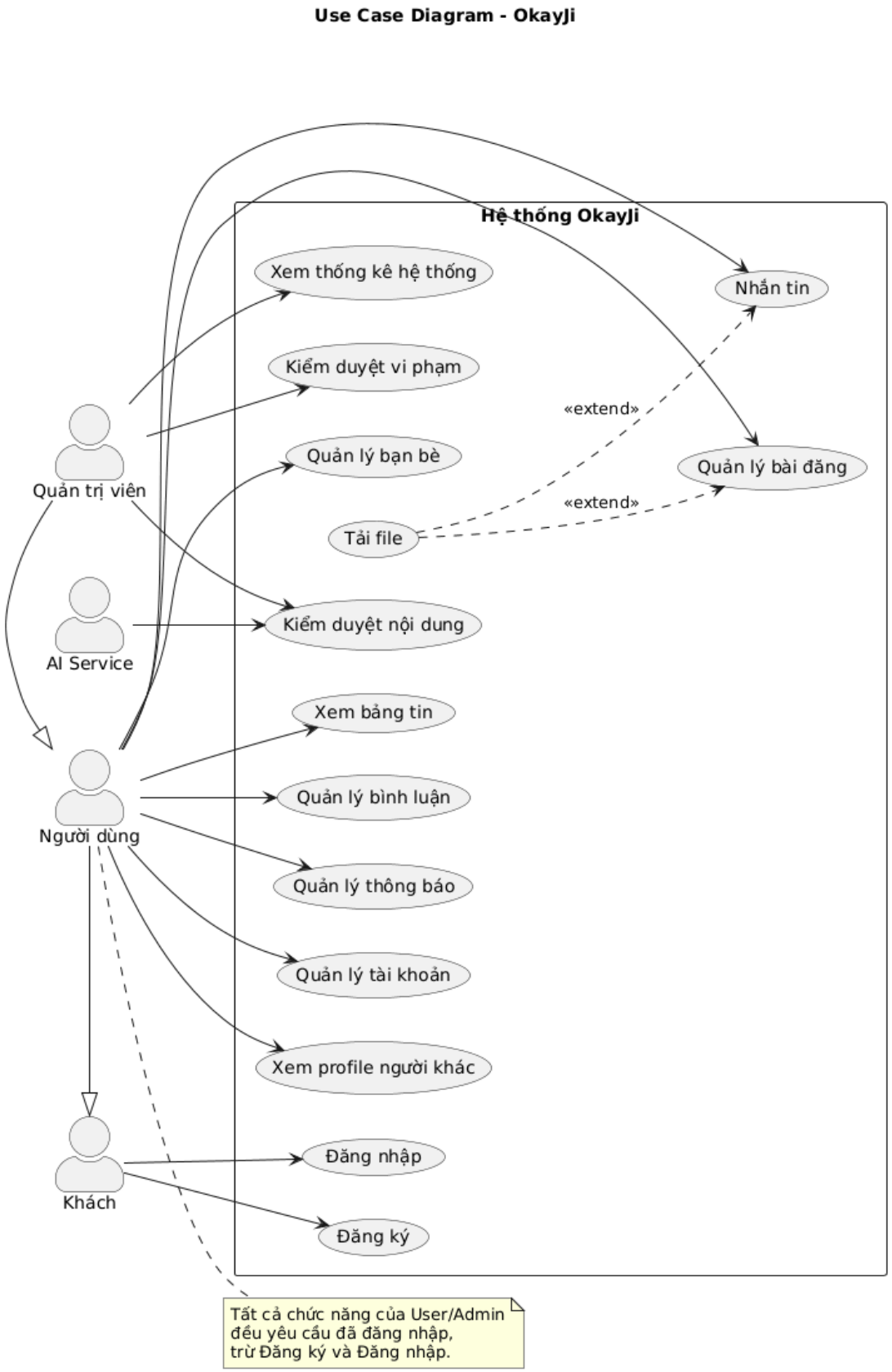
Tương tác	Post	Reaction	Quan hệ 1-n; một bài viết có thể nhận nhiều lượt tương tác, còn mỗi lượt tương tác chỉ gắn với một bài viết
Tương tác	User	Reaction	Quan hệ 1-n; một người dùng có thể tạo nhiều lượt tương tác, còn mỗi lượt tương tác chỉ thuộc về một người dùng
Nhắn tin	User	Chat	Quan hệ 1-n; một người dùng có thể tạo nhiều cuộc trò chuyện, còn mỗi cuộc trò chuyện chỉ có một người tạo
Nhắn tin	Chat	ChatMember	Quan hệ 1-n; một cuộc trò chuyện có thể có nhiều thành viên tham gia, còn mỗi bản ghi thành viên chỉ thuộc về một cuộc trò chuyện
Nhắn tin	User	ChatMember	Quan hệ 1-n; một người dùng có thể tham gia nhiều cuộc trò chuyện, còn mỗi bản ghi trong *chat_member* chỉ ứng với một người dùng

Nhắn tin	Chat	Message	Quan hệ 1-n; một cuộc trò chuyện có thể chứa nhiều tin nhắn, còn mỗi tin nhắn chỉ thuộc về một cuộc trò chuyện
Nhắn tin	User	Message	Quan hệ 1-n; một người dùng có thể gửi nhiều tin nhắn, còn mỗi tin nhắn chỉ do một người gửi
Kiểm duyệt	ModerationJob	ModerationResult	Quan hệ 1-n; một công việc kiểm duyệt có thể sinh ra nhiều kết quả kiểm duyệt, còn mỗi kết quả chỉ thuộc về một công việc
Kiểm duyệt	User	Report	Quan hệ 1-n; một người dùng có thể gửi nhiều báo cáo vi phạm, còn mỗi báo cáo vi phạm chỉ được gửi từ một người dùng
Thông báo	User	Notification	Quan hệ 1-n; một người dùng có thể nhận nhiều thông báo, còn mỗi thông báo chỉ thuộc về một người dùng

2.3. Entity-Relationship Diagram (ERD)



2.4. Use Case Diagram



2.5. Đặc tả 2 chức năng chính

2.5.1. Chức năng tạo bài đăng

Tổng quan: Chức năng tạo bài đăng cho phép người dùng chia sẻ thông tin, cảm xúc bao gồm nội dung văn bản và các tệp đính kèm như hình ảnh hoặc video. Điểm đặc biệt của hệ thống là bài đăng sau khi được tạo sẽ không hiển thị ngay lập tức. Thay vào đó, bài đăng được khởi tạo với trạng thái PENDING (chờ duyệt) và được đưa vào một tiến trình kiểm tra tự động chạy ngầm nhằm phát hiện ảnh, video, ngôn từ vi phạm trước khi chuyển sang trạng thái hiện thị trên bảng tin là PUBLISHED hoặc từ chối hoàn toàn là REJECTED.

Công nghệ sử dụng

Front-end:

React 19 và Vite để xây dựng giao diện phía Client (CreatePost.jsx).

Sử dụng Axios để gọi APIs và tương tác RESTful với Backend.

Các Icons được cung cấp bởi Lucide React.

Back-end:

Java 21 kết hợp cùng Spring Boot 4 (WebMVC, Data JPA).

Spring Security và JWT phục vụ việc xác thực phiên người dùng.

Spring AI: Tích hợp với OpenAI để xây dựng bộ đánh giá nội dung nhằm tự động kiểm tra văn bản, hình ảnh, video của người dùng.

Database:

MySQL: Hệ quản trị CSDL quan hệ dùng lưu trữ Metadata bài viết, người dùng, Job kiểm duyệt, ,... thông qua Spring Data JPA.

Amazon S3: Lưu trữ trực tiếp file đa phương tiện (Bằng Java AWS SDK cho Back-end để tương tác cấp quyền).

Kiến trúc

Để tối ưu hóa hiệu suất và băng thông máy chủ, chức năng tạo bài viết áp dụng 2 Pattern kiến trúc chính:

Direct-to-Cloud Upload (Presigned URL Pattern): Thay vì gửi ảnh hay video trực tiếp thông qua Backend gây tốn tài nguyên server, Backend đóng vai trò cấp một token ủy

quyền dùng một lần (S3 Presigned URL). Sau đó, Client/Frontend trực tiếp upload file lên S3. Hỗ trợ Single-part cho các file nhẹ, và Multi-part Uploading chia nhỏ luồng song song hoặc nối tiếp cho các video lớn (<512 MB).

Asynchronous / Event-driven Processing (Chế độ xử lý bất đồng bộ): Thao tác lưu bài đăng thành công được phản hồi cực nhanh cho phía người dùng. Quá trình kiểm duyệt nội dung tốn nhiều thời gian của AI thay vào đó được đưa vào một Background Worker/Job Scheduler (ModerationJob).

Đặc tả chức năng

* Tình huống chính: Tạo bài đăng có tệp đính kèm thành công

1. Người dùng nhập nội dung bài đăng
2. Người dùng chọn tệp đính kèm (ảnh hoặc video)
3. Người dùng nhấn nút đăng
4. Hệ thống kiểm tra và xác nhận bài đăng có tệp đính kèm
5. Hệ thống tải tệp đính kèm lên AWS S3
6. Hệ thống trả về presign URL và public URL (Media URL) của các tệp đính kèm
7. Hệ thống kiểm tra tính hợp lệ về định dạng tệp của các tệp đính kèm
8. Hệ thống lưu nội dung bài đăng và các Media URL vào cơ sở dữ liệu với trạng thái bài đăng là PENDING
9. Hệ thống tạo và lưu Moderation Job (Job kiểm duyệt)
10. Hệ thống làm mới trang bảng tin
11. Hệ thống thông báo tạo bài đăng thành công
12. Kết thúc chức năng tạo bài đăng

* Tình huống thay thế 1: Tạo bài đăng không có tệp đính kèm thành công

1. Người dùng nhập nội dung bài đăng
2. Người dùng nhấn nút đăng
3. Hệ thống kiểm tra và xác nhận bài đăng không có tệp đính kèm
4. Hệ thống lưu nội dung bài đăng vào cơ sở dữ liệu với trạng thái bài đăng là PENDING
5. Hệ thống tạo và lưu Moderation Job (Job kiểm duyệt)

6. Hệ thống làm mới trang bảng tin
7. Hệ thống thông báo tạo bài đăng thành công
8. Kết thúc chức năng tạo bài đăng

* Tình huống thay thế 2: Tạo bài đăng thất bại do để trống nội dung

1. Người dùng chọn tệp đính kèm (ảnh hoặc video)
2. Người dùng nhấn nút đăng
3. Hệ thống báo lỗi bài đăng không có nội dung
4. Kết thúc chức năng tạo bài đăng

* Tình huống thay thế 3: Tạo bài đăng thất bại do định dạng tệp không hợp lệ

1. Người dùng chọn tệp đính kèm không phải là ảnh hoặc video
2. Hệ thống báo lỗi và yêu cầu chọn tệp đính kèm là ảnh hoặc video
3. Kết thúc chức năng tạo bài đăng

2.5.2. Chức năng kiểm duyệt nội dung tự động bằng AI

Tổng quan: Trong hệ thống mạng xã hội OkayJi, chức năng Kiểm duyệt nội dung tự động bằng AI đóng vai trò như một màng lọc tự động giúp phát hiện và ngăn chặn kịp thời các nội dung độc hại, vi phạm tiêu chuẩn cộng đồng (ngôn từ kích động, bạo lực, nhạy cảm...). Thay vì kiểm duyệt thủ công toàn bộ khối lượng dữ liệu khổng lồ do người dùng tạo ra (User-Generated Content), hệ thống sẽ tạo các truy vấn ngầm (background jobs) để gửi nội dung văn bản, hình ảnh, cũng như video qua model AI đánh giá. Các bài viết an toàn sẽ được tự động chuyển sang trạng thái xuất bản, trong khi những mục vi phạm sẽ bị khóa và thông báo gửi đến người dùng.

Công nghệ sử dụng:

- Spring AI để giao tiếp trực tiếp với OpenAI API (model *omni-moderation-latest*), giúp phân tích đa phương thức (Text, Image).
- FFmpeg để trích xuất khung hình (frames) từ video, giải quyết bài toán AI chưa hỗ trợ nhận diện video trực tiếp.
- Cloud Storage: AWS SDK để lưu trữ file tạm cũng như đẩy các frames trích xuất lên bằng Presigned URL nhằm tạo link truy xuất an toàn cho OpenAI API đánh giá.
- Lập lịch & Xử lý nền (Background Processing): Sử dụng Spring Scheduler (Cơ chế

Cron Job) kết hợp annotations *@Retryable* xử lý lỗi khi gọi third-party services.

- Giao tiếp thời gian thực (Real-time): Sử dụng Spring WebSocket (STOMP) thông báo (Push notification) ngay lập tức tới client của người dùng khi bài đăng/bình luận của họ bị vô hiệu hóa.

Kiến trúc: Quy trình được điều phối bởi 4 thành phần chính trong package *com.okayji.moderation.service*:

- ModerationJobScheduler: Lập lịch định kỳ truy xuất DB để tìm các tác vụ (Job) đang chờ kiểm duyệt.
- ModerationJobExecutor: Thực thi một tác vụ kiểm duyệt, quản lý số lần thử lại (Retry) khi gặp lỗi.
- ModerationOrchestrator: Đóng vai trò bộ não điều phối (Orchestrator), chỉ định chiến lược kiểm duyệt tùy thuộc vào loại đối tượng (POST hay COMMENT).
- ModerationService: Nơi trực tiếp đóng gói payload và gửi REST request tới OpenAPI, S3 và trích xuất Video Frame.

Luồng hoạt động (Workflow):

1. **Khởi động và thu thập tác vụ (Cron Job):**

- EventListener lắng nghe sự kiện khởi động hệ thống để backoff các job đang xử lý dở ở lần chạy trước. Tự động đánh dấu các job đang ở trạng thái PROCESSING về lại trạng thái PENDING.
- Mỗi tác vụ bài đăng (POST) hoặc bình luận (COMMENT) mới khi tạo ra sẽ được đính kèm một ModerationJob với trạng thái PENDING lưu trong DB. Cứ sau mỗi 30s, Hệ thống (System Cron) kích hoạt *ModerationJobScheduler.processPendingJobs()* để lấy danh sách các tác vụ PENDING.

2. **Execution & Retry:**

- Duyệt qua danh sách các Job, gửi lệnh đến ModerationJobExecutor để khoá (Lock) trạng thái tác vụ thành PROCESSING và tăng biến đếm retryCount.
- Hệ thống phân luồng dựa vào phân loại cụ thể của đối tượng (TargetType: POST hoặc COMMENT).

3. **Phân tích & Kiểm duyệt Đa phương tiện:** Hệ thống cho phép quét đa tầng, dừng lại ngay nếu phát hiện cấp độ vi phạm nghiêm trọng nhằm tối ưu chi phí API.

   Trường hợp 1: Kiểm duyệt Bài viết (POST) ModerationOrchestrator phụ trách chia nhỏ kiểm duyệt theo thành phần bài đăng:

   - Đối với Văn bản (Text): Gửi nội dung thô qua ModerationService, kết nối OpenAI API để nhận điểm số (Scores/Flags) về mức độ vi phạm.
   - Đối với Hình ảnh (Image): Nếu bước lọc văn bản không vi phạm nghiêm trọng (Chưa bị BLOCK), hệ thống gửi tiếp đường dẫn URL của hình ảnh lên OpenAI API đánh giá.
   - Đối với Video: ModerationService gọi AWS S3 tải tạm video về server. Sử dụng FFmpeg để cắt video thành nhiều khung hình (Frames) tĩnh. Vòng lặp tự động tải ngược từng frame lên AWS S3 dưới dạng Presigned URL ngắn hạn. Gọi OpenAI API duyệt qua từng URL chứa frame ảnh. Xoá dọn dẹp các tệp tạm trên server và S3 sau khi có kết quả để tối ưu lưu trữ.

   Trường hợp 2: Kiểm duyệt Bình luận (COMMENT): Trích xuất dữ liệu, chỉ thực hiện gọi OpenAI API qua module Text.

4. **Đánh giá Tổng hợp và Cập nhật Trạng thái:** ModerationOrchestrator tổng hợp các phản hồi và lưu thành bản ghi ModerationResult vào DB.

   Quyết định cho POST: Quy kết trạng thái của bài viết thành 1 trong 3 trạng thái: PUBLISHED: Hợp lệ, hiển thị công khai; UNDER_REVIEW: Thuộc vùng xám (nghi ngờ vi phạm), tạm thời đưa về diện Chờ phê duyệt thủ công từ Admin; REJECTED: Chắc chắn vi phạm, bài viết bị khóa hoàn toàn.

   Quyết định cho COMMENT: Nếu phát hiện vi phạm, comment sẽ bị tiến hành Xoá khỏi (delete) Database vĩnh viễn thay vì ẩn đi.

5. **Phản hồi Thời gian thực (Real-time Notification):**

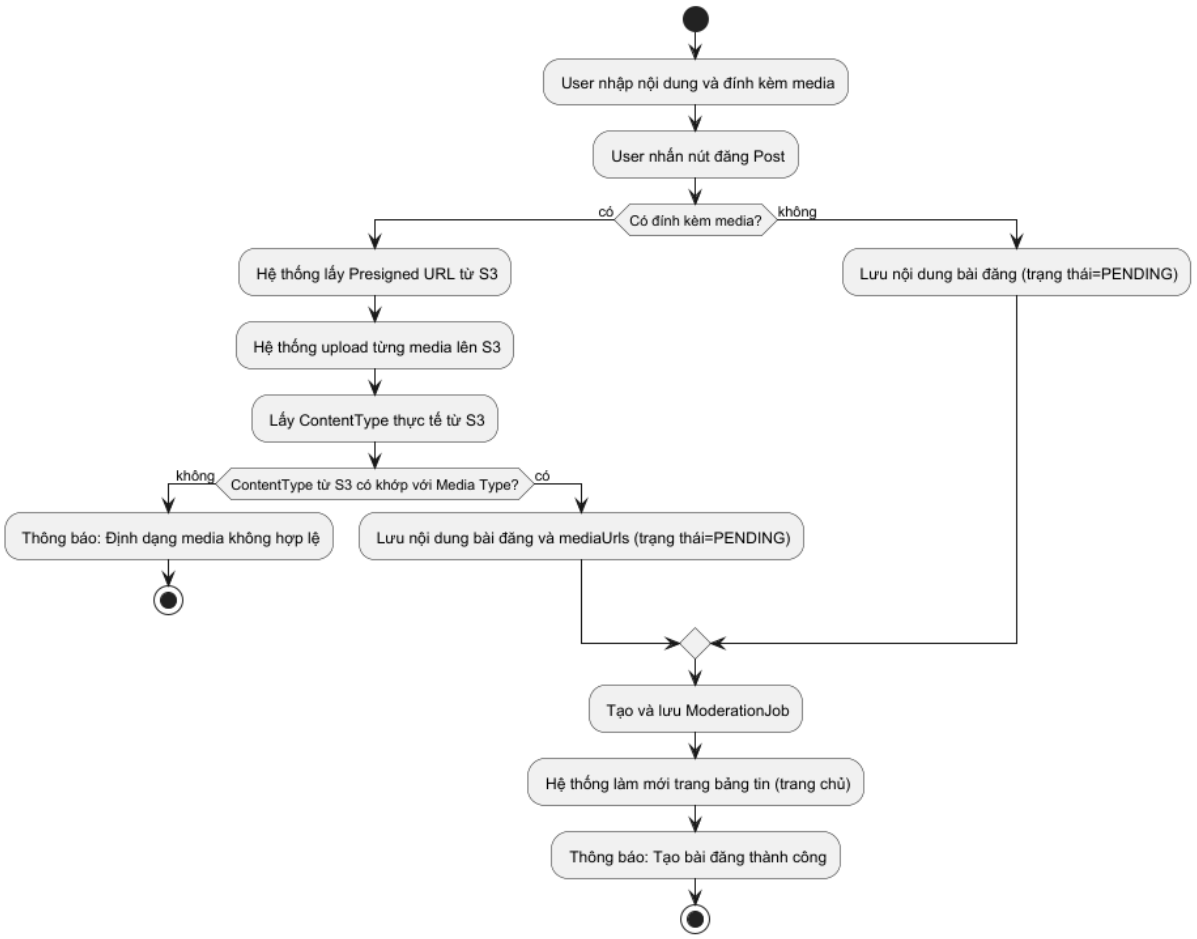
   Đối với mọi hành động phạt (Bài đăng bị REJECTED/UNDER_REVIEW hoặc Bình luận bị Xóa), hệ thống gọi trực tiếp NotificationService để gửi gói tin thông báo qua kênh WebSocket thẳng tới máy khách (Client) để báo cho người dùng lý do vi phạm bài đăng của họ một cách công khai, minh bạch.

6. **Hoàn tất & Cơ chế chịu lỗi (Fault Tolerance):**

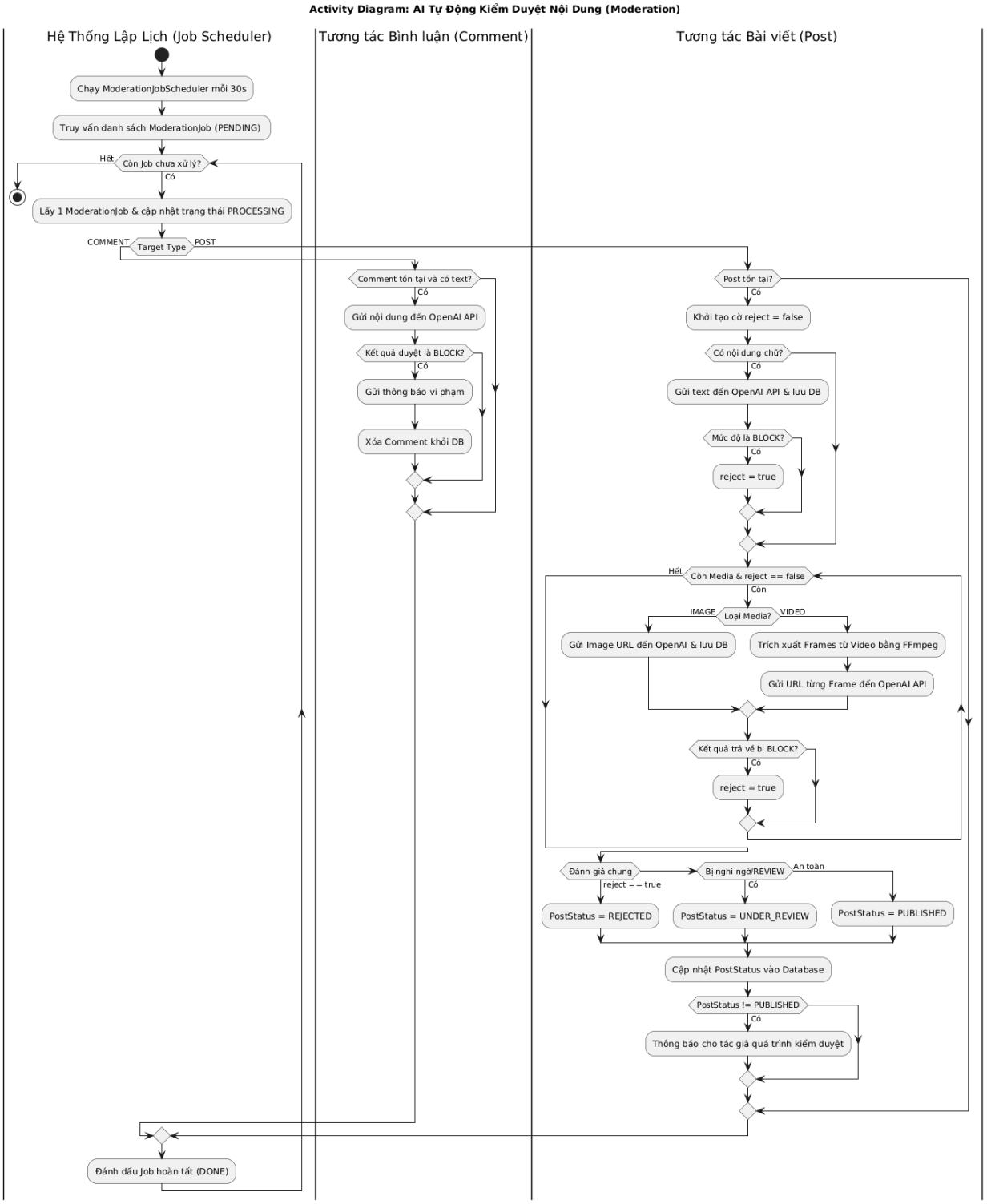
- Cuối mỗi chu trình tác vụ, trạng thái các Job được cập nhật thành DONE.
- Nếu quá trình giao tiếp bị rớt mạng hay API quá tải (Exception), Executor kích hoạt *@Retryable* lặp lại thêm 2 lần nữa. Khi gọi lại thất bại sau 3 lần, cơ chế *recover(Exception)* sẽ giáng job về trạng thái FAILED nếu đã đạt max_retries hoặc về lại PENDING để giữ an toàn dữ liệu chờ đợt xử lý quét lại, tránh thất thoát yêu cầu.

2.6. Activity Diagram

2.6.1. Chức năng tạo bài viết

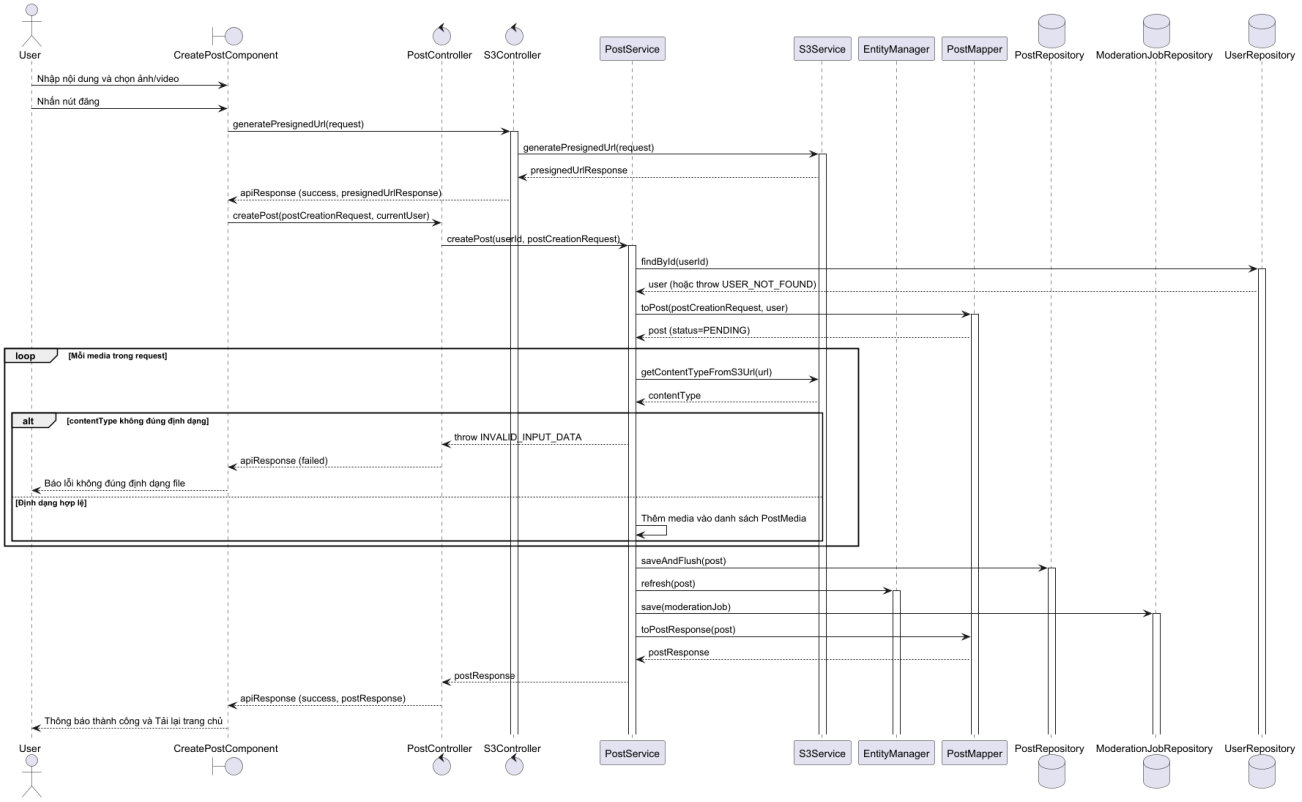


2.6.2. Chức năng kiểm duyệt nội dung tự động bằng AI

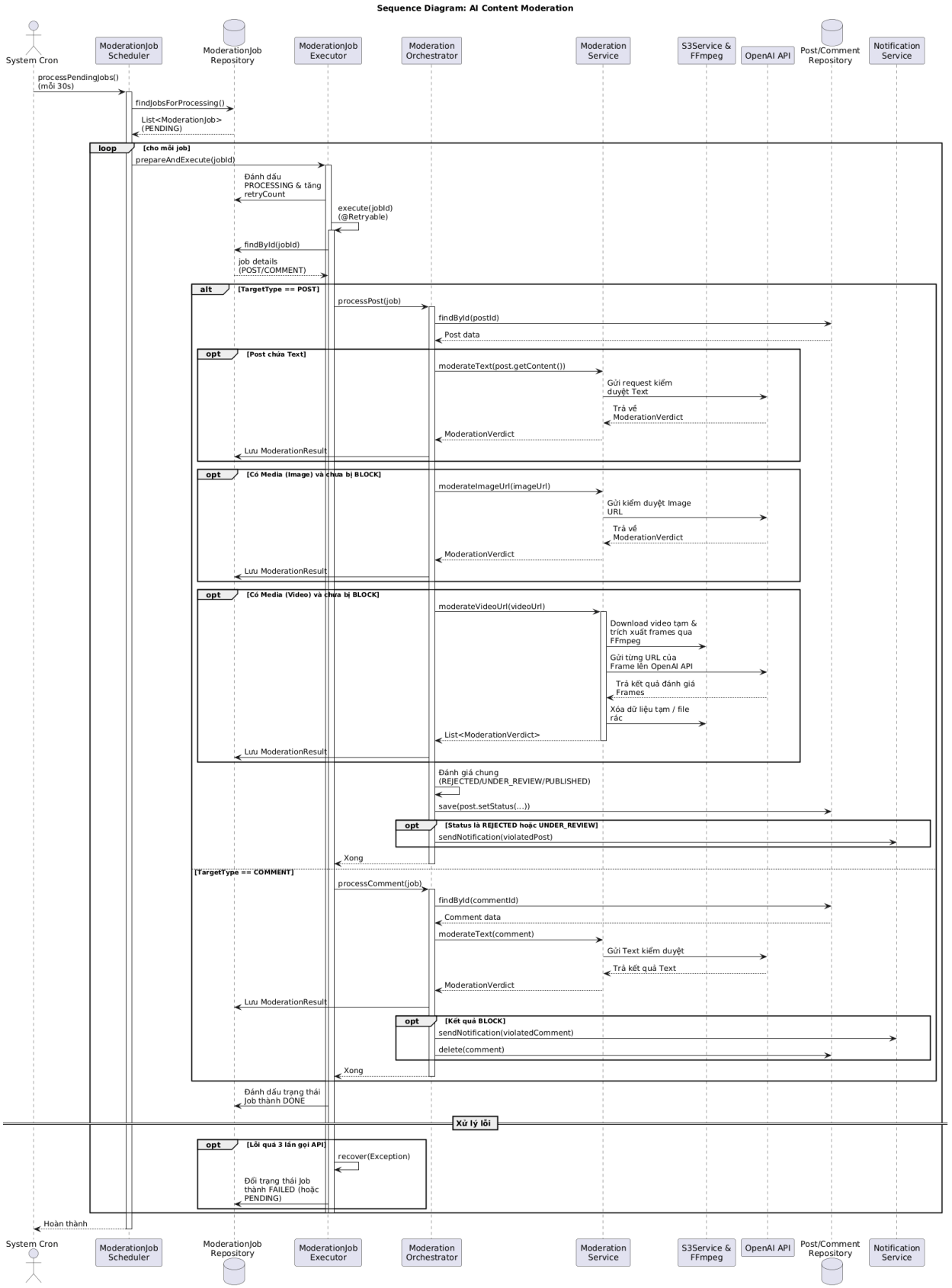


2.7. Sequence Diagram

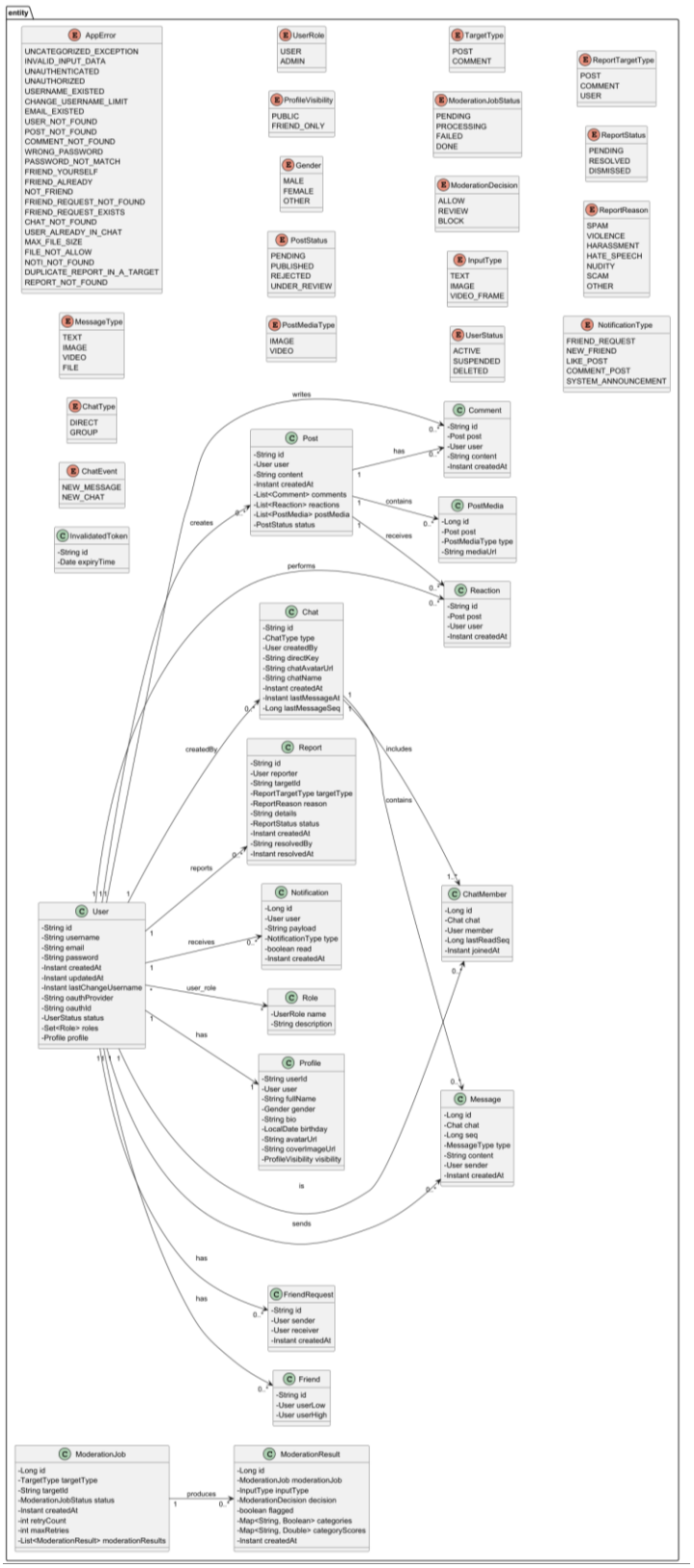
2.7.1. Chức năng tạo bài viết

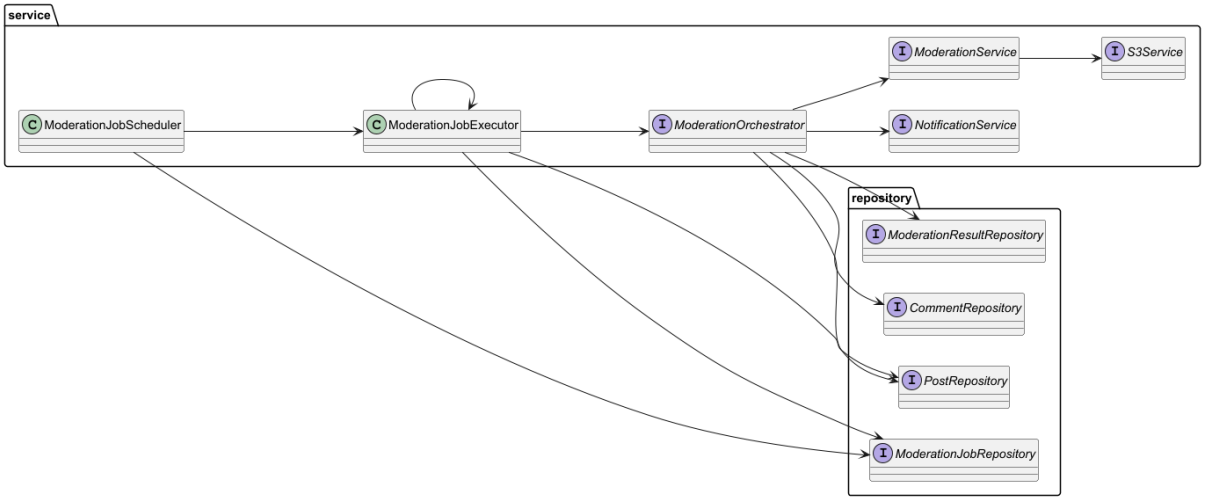


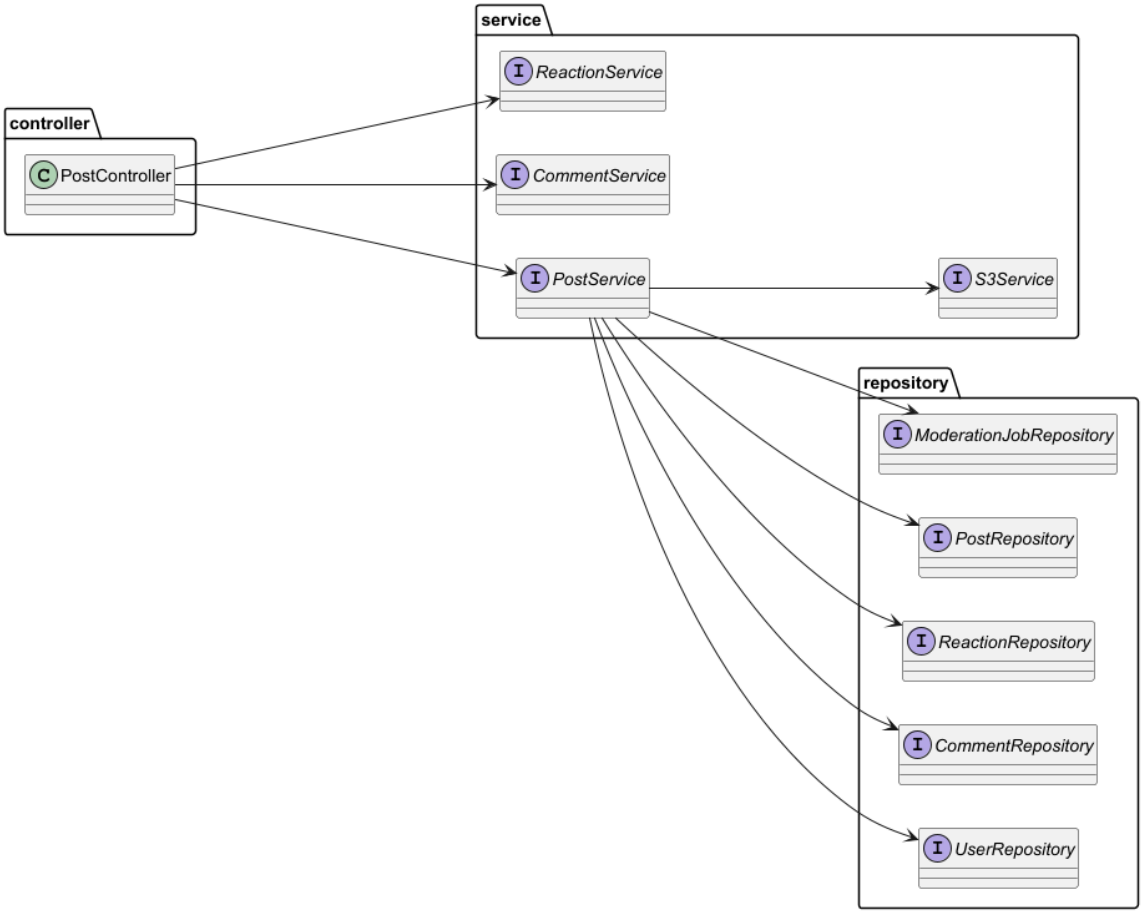
2.7.2. Chức năng kiểm duyệt nội dung tự động bằng AI



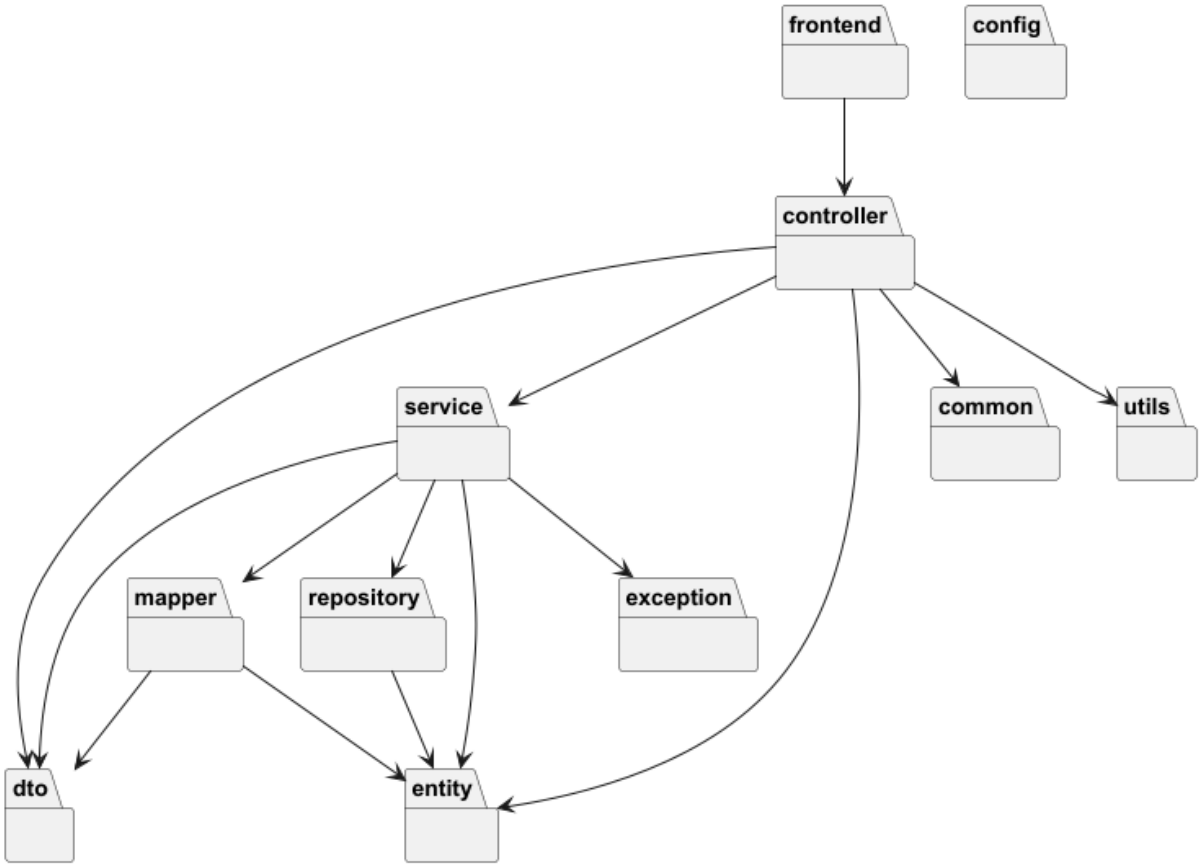
2.8. Class Diagram




service
ModerationService
S3Service
ModerationJobScheduler
ModerationJobExecutor
ModerationOrchestrator
NotificationService
repository
ModerationResultRepository
CommentRepository
PostRepository
ModerationJobRepository


service
ReactionService
controller
PostController
CommentService
PostService
S3Service
repository
ModerationJobRepository
PostRepository
ReactionRepository
CommentRepository
UserRepository

2.9. Package Diagram



CHƯƠNG 3: KIẾN TRÚC HỆ THỐNG

3.1. Tổng quan

Hệ thống mạng xã hội OkayJi được xây dựng và tổ chức theo mô hình kiến trúc Client-Server hiện đại, kết hợp với các dịch vụ bổ trợ bên thứ ba (Third-party Services) để tối ưu hóa việc lưu trữ và xử lý thông tin phức tạp. Tổng quan hệ thống bao gồm các thành phần chính sau:

- **Môi trường Front-end (Client-side):** Đóng vai trò là lớp Giao diện người dùng (Presentation Layer). Front-end được xây dựng bằng ReactJS, giao tiếp với Back-end thông qua các RESTful API chuẩn HTTP cho các tác vụ lấy dữ liệu tĩnh/đồng bộ, và sử dụng giao thức WebSocket (STOMP protocol) cho các tác vụ theo thời gian thực (Real-time) như nhắn tin và nhận thông báo.
- **Môi trường Back-end (Server-side):** Đóng vai trò là Hệ thống trung tâm (Business Logic Layer & API Gateway). Xây dựng trên nền tảng Java Spring Boot, Back-end chịu trách nhiệm xử lý các nghiệp vụ cốt lõi, xác thực và phân quyền người dùng (JWT, Spring Security), điều phối các luồng tin nhắn WebSocket, và đảm bảo tính thống nhất dữ liệu trước khi đẩy xuống cơ sở dữ liệu.
- **Cơ sở dữ liệu (Data Layer):** Sử dụng hệ quản trị cơ sở dữ liệu quan hệ MySQL. Dữ liệu được truy xuất thông qua Spring Data JPA/Hibernate (ORM technology).
- **Các Dịch vụ liên kết bên ngoài (External Services):**
  - **AWS S3 (Amazon Simple Storage Service):** Quản lý, lưu trữ độc lập các tệp tĩnh có dung lượng lớn được người dùng đăng tải (hình ảnh, video, tệp văn bản).
  - **OpenAI API (Moderation Model):** Tích hợp qua Spring AI, đóng vai trò như một bộ lọc thông minh tự động dự đoán và đánh giá nội dung do người dùng đăng tải để từ chối các thông tin vi phạm chính sách hiển thị.
  - **FFmpeg:** Một công cụ chạy trực tiếp trên máy chủ để xử lý nén, chuyển đổi định dạng video.

Luồng dữ liệu tổng quát (Data Flow): Client (Trình duyệt) gửi request (HTTP / WebSocket) chứa JWT token (nếu yêu cầu quyền) -> Filter xác thực tại Back-end kiểm tra hợp lệ -> Controllers điều phối request đến Services tương ứng -> Services thực thi

nghiệp vụ, gọi ra External Services (AWS, OpenAI) nếu cần, hoặc tương tác với Repositories -> Repositories truy cập MySQL Database -> Trả về kết quả cho Services để đóng gói thành chuẩn ApiResponse -> Phản hồi lại UI Client.

3.2. Kiến trúc Back-end

Kiến trúc bao gồm 3 thành phần chính:

- Controller (Giao tiếp & Điều phối):
  - Đảm nhận việc tiếp nhận các HTTP Request và WebSocket STOMP messages.
  - Thực hiện validation (kiểm tra tính hợp lệ) ban đầu của dữ liệu trích xuất từ Payload/DTO request.
  - Điều phối luồng công việc xuống tầng Service, sau đó bọc (wrap) kiểu dữ liệu trả về bằng class cấu trúc đồng nhất ApiResponse.
- Service (Xử lý Nghiệp vụ Logic):
  - Đây là trái tim của hệ thống Back-end. Các file tại tầng này đều được triển khai dựa trên các Interfaces.
  - Thực thi logic tính toán, xác thực các quy tắc nghiệp vụ rẽ nhánh, phối hợp với các Service của module khác.
  - Gọi và điều phối các dịch vụ từ AWS S3 (upload file) hay Modertion API (hệ thống kiểm duyệt nội dung AI).
- Repository (Tương tác Cơ sở dữ liệu):
  - Kế thừa từ các interface của Spring Data JPA, đóng vai trò ánh xạ cấu trúc Đối tượng vào bảng dữ liệu (ORM - Object Relational Mapping).
  - Tầng này cho phép ứng dụng giao tiếp an toàn với cơ sở dữ liệu MySQL mà không cần viết quá nhiều query gốc.

Tổ chức theo Module: Thay vì gom chung tất cả logic vào các package phẳng, mã nguồn Back-end được gom thành nhiều package:

```
back-end/
├── src/
│   ├── main/
│   │   ├── java/com/okayji/
│   │   │   ├── OkayjiApplication.java
│   │   │   ├── admin/
│   │   │   ├── identity/
│   │   │   ├── feed/
│   │   │   ├── chat/
│   │   │   ├── notification/
│   │   │   ├── moderation/
│   │   │   ├── report/
│   │   │   ├── file/
│   │   │   ├── config/
│   │   │   ├── common/
│   │   │   ├── exception/
│   │   │   └── mapper/
│   │   └── resources/
│   │       └── application.yaml
│   └── test/
├── pom.xml
├── mvnw
├── mvnw.cmd
└── README.md
```

- identity: Xử lý đăng ký, đăng nhập (JWT generation/validation), profile cá nhân và cấp quyền (RBAC).
- feed: Chịu trách nhiệm hiển thị/đăng tải bài viết, bình luận, trạng thái thả tim, kết bạn.
- chat & notification: Tách biệt việc xử lý event stream real-time. Tích hợp STOMP Message Broker cho các hội thoại 1-1, chat nhóm và dispatch hệ thống thông báo đa tương tác.
- moderation: Quản lý tác vụ kiểm duyệt theo dạng hàng đợi phi đồng bộ. ModerationJobScheduler định kỳ chạy tự động kéo các nội dung cần duyệt và gọi ModerationOrchestrator giao tiếp với OpenAI tiến hành phân tích nội dung vi phạm.
- report: Xử lý việc tạo các báo cáo về bài đăng, bình luận, người dùng
- admin: Nghiệp vụ dành riêng cho quản trị viên, nơi thống kê số liệu tổng quan hệ

thống và xét duyệt thủ công các nội dung báo cáo hoặc bài đăng chưa xác định rõ thuộc tính.

- file: Cấu trúc upload tệp phân thân đa phần, trả mã định danh từ AWS S3 bucket.

3.3. Kiến trúc Front-end

Kiến trúc Front-end của OkayJi được xây dựng dựa trên hệ sinh thái ReactJS kết hợp build-tool Vite nhằm đảm bảo khả năng mở rộng, dễ quản lý state phức hợp và mang lại tốc độ phản hồi phía Client tối ưu.

```
front-end/
├── public/
├── src/
│   ├── api/
│   ├── components/
│   ├── context/
│   ├── pages/
│   ├── styles/
│   ├── utils/
│   ├── App.jsx
│   ├── App.css
│   ├── index.css
│   └── main.jsx
├── index.html
├── vite.config.js
├── eslint.config.js
├── package.json
├── package-lock.json
└── README.md
```

Kiến trúc Thành phần: Front-end phân tách cấu trúc hiển thị UI thành mô hình Pages + Component:

- Pages (src/pages): Là những Container components, được map với một Route cụ thể qua React Router DOM tương đương với từng màn hình hiển thị (Home, Profile, PostDetail, Chat, ...).
- Components (src/components): Đóng vai trò Dumb-components (Presentational Components) có tính tái sử dụng cao, chỉ nhận Props đầu vào và hiển thị thuần tuý. Chúng được chia làm nhiều tệp nhỏ lẻ như (PostCard, ChatMessage, CreatePost, ...) để tạo nên kiến trúc lắp ghép chuẩn.

State Management Strategy: Hệ thống sử dụng linh hoạt dựa trên phạm vi hoạt động của Data:

- Global State: Sử dụng React Context API cho dữ liệu xuyên suốt toàn ứng dụng như ChatContext (Duy trì và cung cấp phương thức gửi/nhận tin nhắn WebSocket), NotificationContext (Real-time updates cho panel thông báo).
- Local State: Sử dụng React Hooks (useState, useReducer) cho các trạng thái giao diện ngắn hạn bên trong các component đơn lẻ như Input value trang đăng nhập, danh sách loadmore của Feed.
- Session State: Sử dụng localStorage trên trình duyệt lưu trữ JWT Token đảm bảo việc "Nhớ đăng nhập" qua mỗi lần tải lại trang.

API Layer: Do đặc thù Front-end cần thực hiện nhiều tương tác liên tục, mọi kết nối HTTP Restful tới Back-end được cô lập hoàn toàn vào lớp src/api/:

- Trọng tâm lớp này là module apiClient.js xây dựng bằng thư viện Axios, đóng vai trò như một Pipeline/Interceptor tập trung. Mọi Request gửi đi sẽ tự động được kiểm tra và đính JWT Token vào tiêu đề (Header Authorization). Mọi Response trả về mang mã lỗi 403/401 sẽ tự động trigger cơ chế clear local storage và redirect hệ thống ra màn hình (Login) giúp bảo mật triệt để ở môi trường User.
- Bên cạnh các API được cấu trúc theo tên dịch vụ như feedService, authService, các tương tác yêu cầu băng thông liên tục và đẩy hai chiều (Bi-Directional) sử dụng thư viện @stomp/stompjs chạy qua một fallback bảo mật là sockjs-client giúp ứng dụng duy trì kết nối WebSocket bền vững trên mọi loại môi trường trình duyệt

CHƯƠNG 4: MÔ TẢ SOLID ĐÃ ÁP DỤNG

4.1. Single Responsibility Principle (SRP)

Định nghĩa: Một lớp (class) hoặc một module chỉ nên có duy nhất một lý do để thay đổi, tức là nó chỉ nên đảm nhận đúng một trách nhiệm cụ thể trong hệ thống.

Minh chứng trong hệ thống qua kiến trúc phân tầng 3 lớp:

- Controller Layer: Các Controller (ví dụ PostController, AuthController) chỉ đảm nhận nhiệm vụ làm cầu nối HTTP: tiếp nhận URL route, xác thực định dạng dữ liệu đầu vào (Validation qua @Valid), và trả về định dạng phản hồi chuẩn hóa (ApiResponse). Controller không chứa bất kỳ logic nghiệp vụ phức tạp nào.
- Service Layer: Chỉ chứa các quy tắc nghiệp vụ kinh doanh (Business Logic).
- Repository Layer: Chỉ chứa phần giao tiếp truy vấn với cơ sở dữ liệu MySQL thông qua Spring Data JPA.

4.2. Open/Closed Principle (OCP)

Định nghĩa: Các thực thể phần mềm (Class, Module, Hàm...) nên MỞ để hỗ trợ việc mở rộng (thêm tính năng mới), nhưng phải ĐÓNG đối với việc sửa đổi (không sửa đổi mã nguồn cốt lõi đang chạy ổn định).

Minh chứng trong hệ thống:

- Cơ chế phân phối và quản lý loại Thông báo: Dự án sử dụng Enum NotificationType kết hợp với các lớp Payload mở rộng đa hình (như LikePostPayload, SystemPayload). Khi có yêu cầu thêm mới một loại thông báo (ví dụ: chia sẻ bài viết, tag tên), lập trình viên chỉ việc khai báo thêm hằng số Enum và bổ sung hàm build ở NotificationFactory. Lõi xử lý nghiệp vụ lưu trữ cơ sở dữ liệu và luồng WebSocket phát đi (STOMP) của hệ thống là bất biến (Đóng với sự sửa đổi).
- Kiến trúc AI kiểm duyệt tự động (ModerationOrchestrator): Hệ thống kiểm duyệt tự động bài viết được điều phối qua một ModerationOrchestrator. Nếu trong tương lai dự án có thêm các quy trình kiểm duyệt nội bộ như Bộ lọc từ khóa xấu (Blacklist Filter) hay Kiểm tra vi phạm hình ảnh, nhà phát triển có thể dễ dàng khai báo và chèn thêm Pipeline mới đứng trước hoặc sau bước dùng AI OpenAI. Cấu trúc hiện

tại cho phép plug-in linh hoạt mà không phá vỡ hay phải đập bỏ mã nguồn gốc của hệ thống kiểm duyệt.

4.3. Liskov Substitution Principle (LSP)

Định nghĩa: Các đối tượng của lớp con phải có khả năng thay thế hoàn toàn cho các đối tượng của lớp cha mà không làm thay đổi tính đúng đắn của chương trình gốc.

Minh chứng trong hệ thống: Toàn bộ tầng Business Logic đều sử dụng Interface làm Hợp đồng giao tiếp, ví dụ như Interface ChatService, FriendService, … Các Implementation tương ứng của chúng (chẳng hạn ChatServiceImpl) bảo đảm việc thực thi đúng mọi ràng buộc và quy tắc định sẵn mà hệ thống mong đợi ở tính năng Chat. Việc này cho phép tại tầng Controller, Client luôn giữ tham chiếu đến Interface. Bất kỳ sự thay thế nào bởi Implementation khác lúc runtime (ví dụ sau này thay ChatServiceImpl thành cấu trúc tối ưu chạy bằng dịch vụ hàng đợi KafkaChatServiceImpl), hoặc lúc viết Unit Test thay bằng mô hình giả lập (MockChatService), đều đảm bảo hệ thống vẫn vận hành an toàn và chính xác, không gây lỗi logic cho tầng HTTP bên trên.

4.4. Interface Segregation Principle (ISP)

Định nghĩa: Nên thiết kế nhiều Interface nhỏ, cụ thể phục vụ riêng rẽ cho từng mục đích, thay vì thiết kế một Interface cồng kềnh (Fat Interface) ôm đồm quá nhiều thứ khiến các Client phải triển khai những phương thức mà chúng không bao giờ dùng tới.

Minh chứng trong hệ thống:

- Tại module Feed (Bảng tin), dự án nghiêm khắc từ chối việc hình thành class "thánh" (God Class) như MainFeedService chứa hàng chục tác vụ tổng hợp (Cả Post, Comment, Like, Follow).
- Thay vào đó, API Service được phân nhỏ theo sát các thực thể miền (Domain Entity): Có các Interface độc lập hoàn toàn như PostService (chỉ quản trị bài viết), CommentService (chỉ quản lý bình luận), và FriendService (chỉ quản lý quan hệ kết bạn).
- Lợi ích đem lại: Khi module NotificationController hoặc module ChatController khởi tạo, nó chỉ cần tham chiếu chính xác Interface nào thực sự phát sinh nhu cầu

sử dụng. Điều này làm giảm rõ rệt độ nhiễu mã nguồn, tránh sự phụ thuộc chéo.

4.5. Dependency Inversion Principle (DIP)

Định nghĩa: Module cấp cao không nên phụ thuộc vào các module cấp thấp. Cả hai nên phụ thuộc vào các Abstraction (Interface / Abstract Class). Abstraction không nên phụ thuộc vào chi tiết kỹ thuật/cài đặt. Mà chi tiết cài đặt phải tuân thủ và phụ thuộc theo Abstraction.

Minh chứng trong hệ thống:

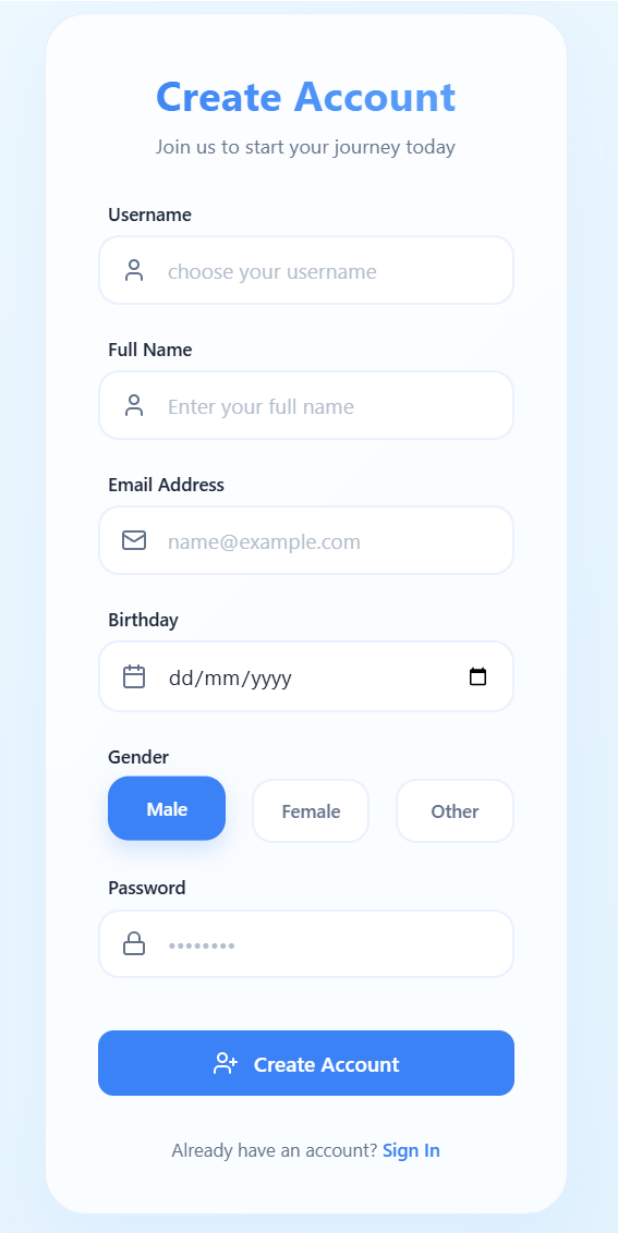
- Tách rời luồng phụ thuộc tại Middleware: Ví dụ: PostController (Module tầng cao phụ trách cấu hình HTTP) đóng vai trò nhận request người dùng. Tuy nhiên, nó tuyệt đối KHÔNG tham chiếu hay tạo trực tiếp một đối tượng mang tính chi tiết bằng lệnh cấp thấp (như new PostServiceImpl()). Controller này chỉ liên kết qua duy nhất lớp Interface trừu tượng PostService.
- Sự ủy quyền cho Spring Inversion of Control (IoC):
  - Thông qua Container đặc biệt của Spring (ApplicationContext) kết hợp @RequiredArgsConstructor (Lombok), nền tảng sẽ tự động quét và đảm nhận toàn bộ việc "Tiêm sự phụ thuộc" (Dependency Injection). Cơ chế này tự động tiêm module cài đặt (ServiceImpl) phù hợp vào Controller hoặc các Service khác lúc khởi động (runtime).
  - Cách tiếp cận này đảo ngược hoàn toàn luồng kiểm soát truyền thống, giúp kiến trúc ứng dụng bị giải phóng hoàn toàn khỏi các logic khởi tạo phức tạp trực tiếp, biến ứng dụng kết nối trở nên lỏng lẻo (loose coupling).

CHƯƠNG 5: GIAO DIỆN VÀ KIỂM THỬ

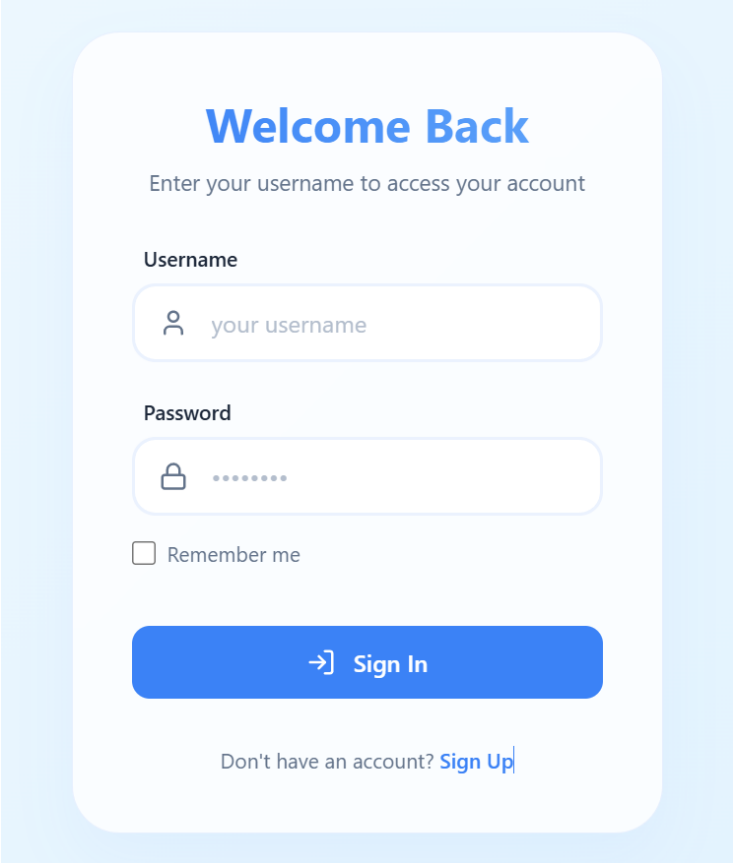
5.1. Giao diện xác thực và quản lý hồ sơ người dùng

Giao diện đăng ký và đăng nhập là điểm bắt đầu của hệ thống, cho phép người dùng tạo tài khoản mới, đăng nhập để nhận JWT Access Token và sử dụng cơ chế "nhớ đăng nhập" trong khoảng thời gian được cấu hình. Giao diện này được thiết kế theo hướng đơn giản, dễ thao tác và phù hợp với yêu cầu về tính trực quan của một nền tảng mạng xã hội.

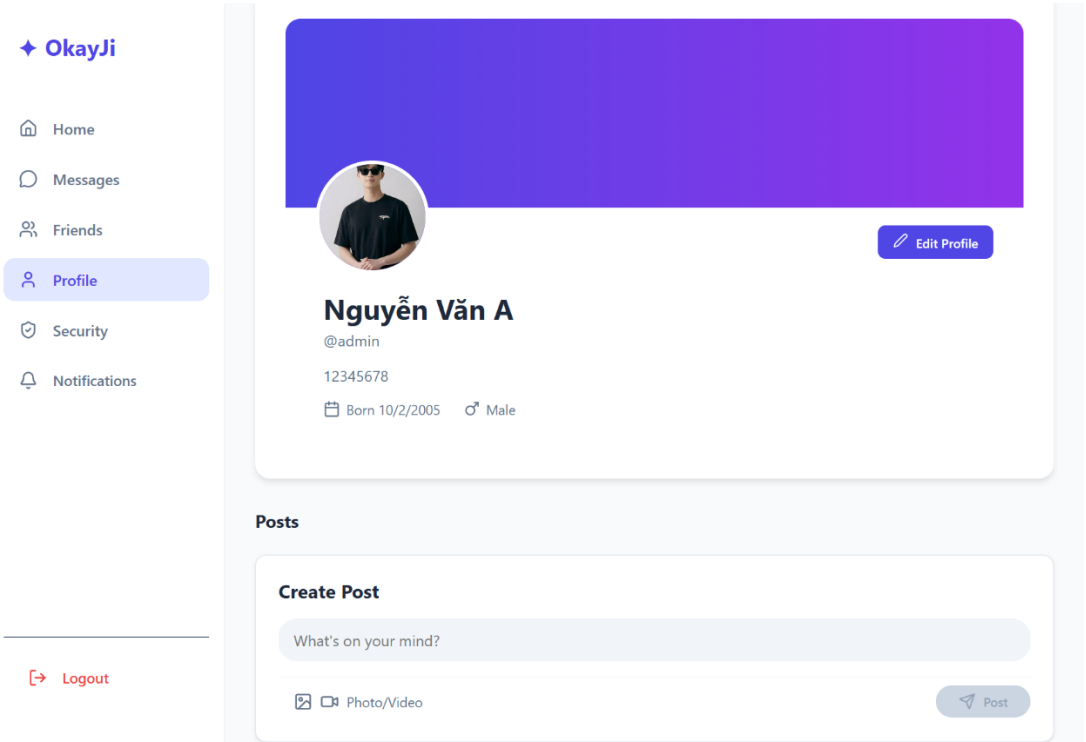
Giao diện đăng ký:

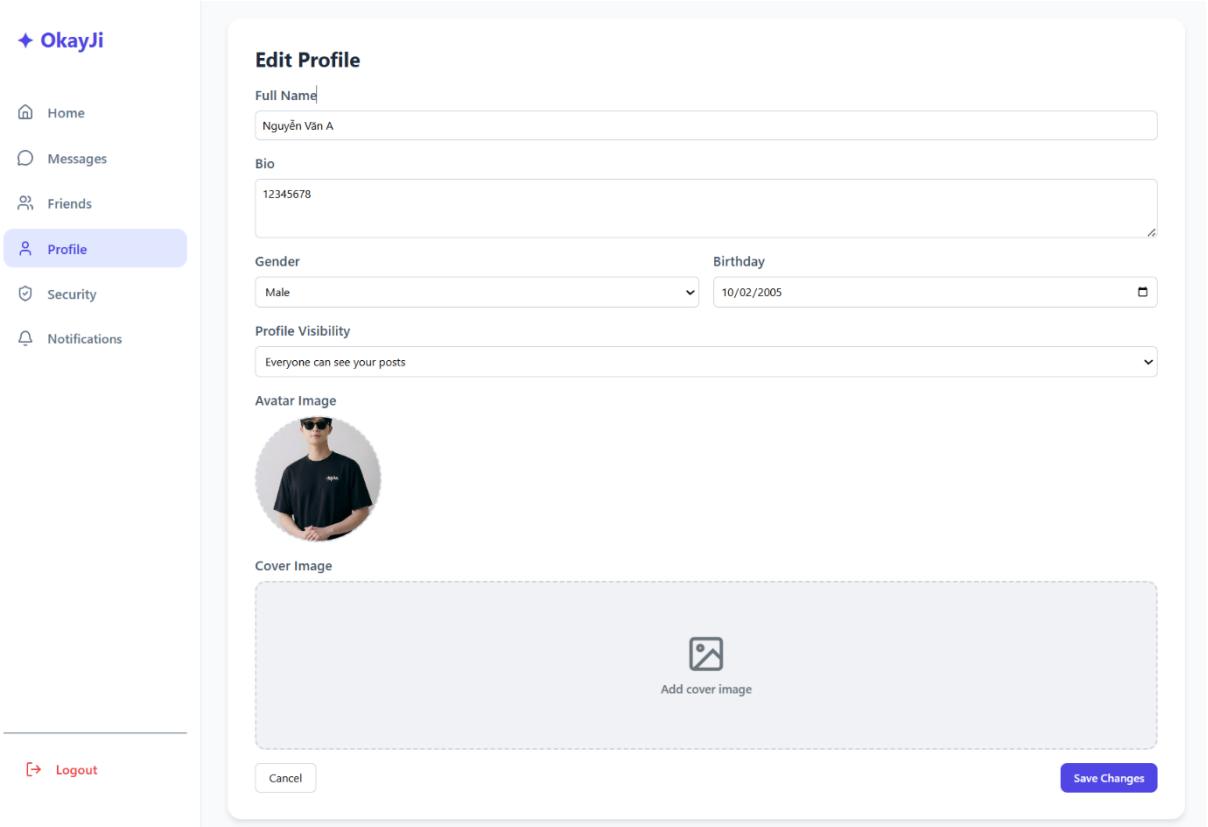


Giao diện đăng nhập:



Giao diện trang cá nhân và cập nhật trang cá nhân:





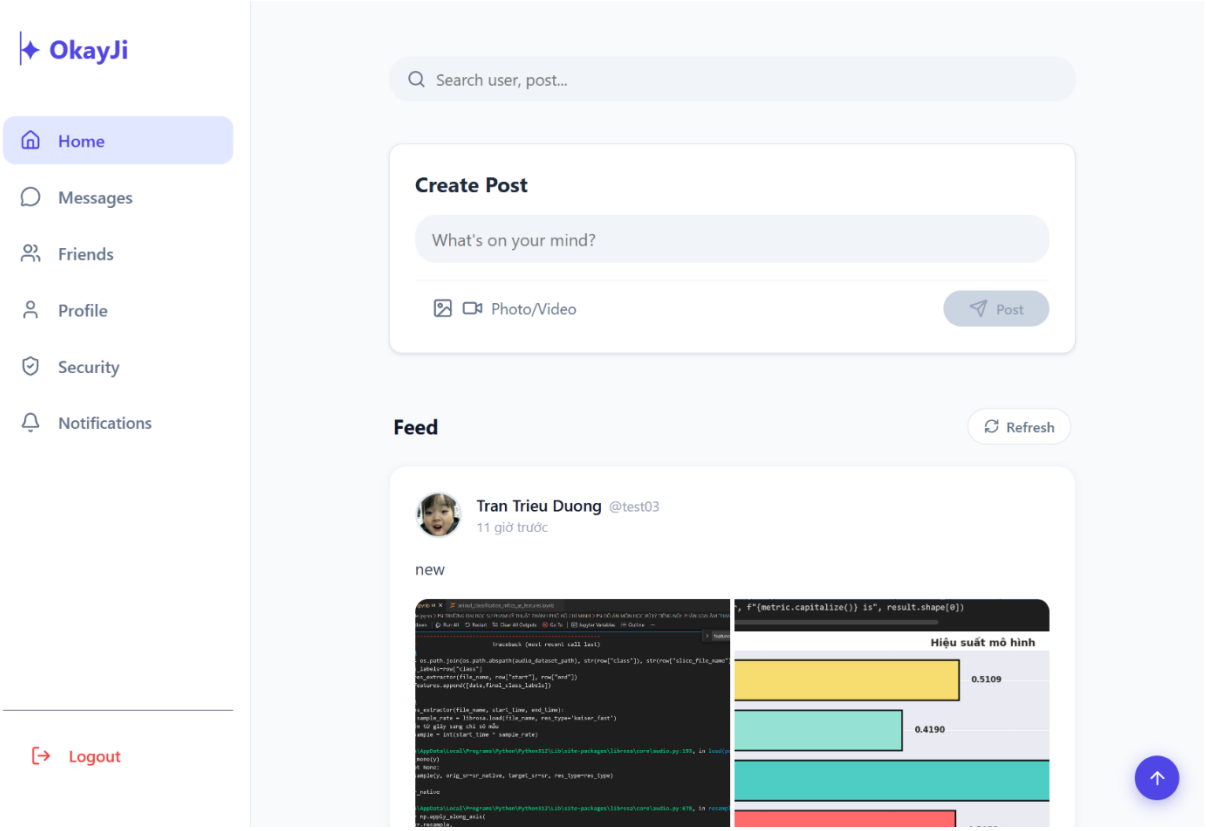
Sau khi đăng nhập thành công, người dùng có thể truy cập vào hồ sơ cá nhân để cập nhật các thông tin như ảnh đại diện, tiểu sử, giới tính, ngày sinh và quyền riêng tư hồ sơ. Hệ thống hỗ trợ thiết lập chế độ hiển thị hồ sơ như công khai hoặc chỉ bạn bè.

Giao diện hồ sơ được tổ chức theo hướng tập trung vào thông tin cá nhân và các thao tác cập nhật cơ bản, giúp người dùng mới có thể nhanh chóng làm quen với việc chỉnh sửa hồ sơ của mình. Việc tách biệt phần xác thực và phần quản lý hồ sơ cũng phù hợp với cách tổ chức module của hệ thống ở phía back-end, nơi nhóm identity phụ trách đăng ký, đăng nhập, profile cá nhân và cấp quyền.

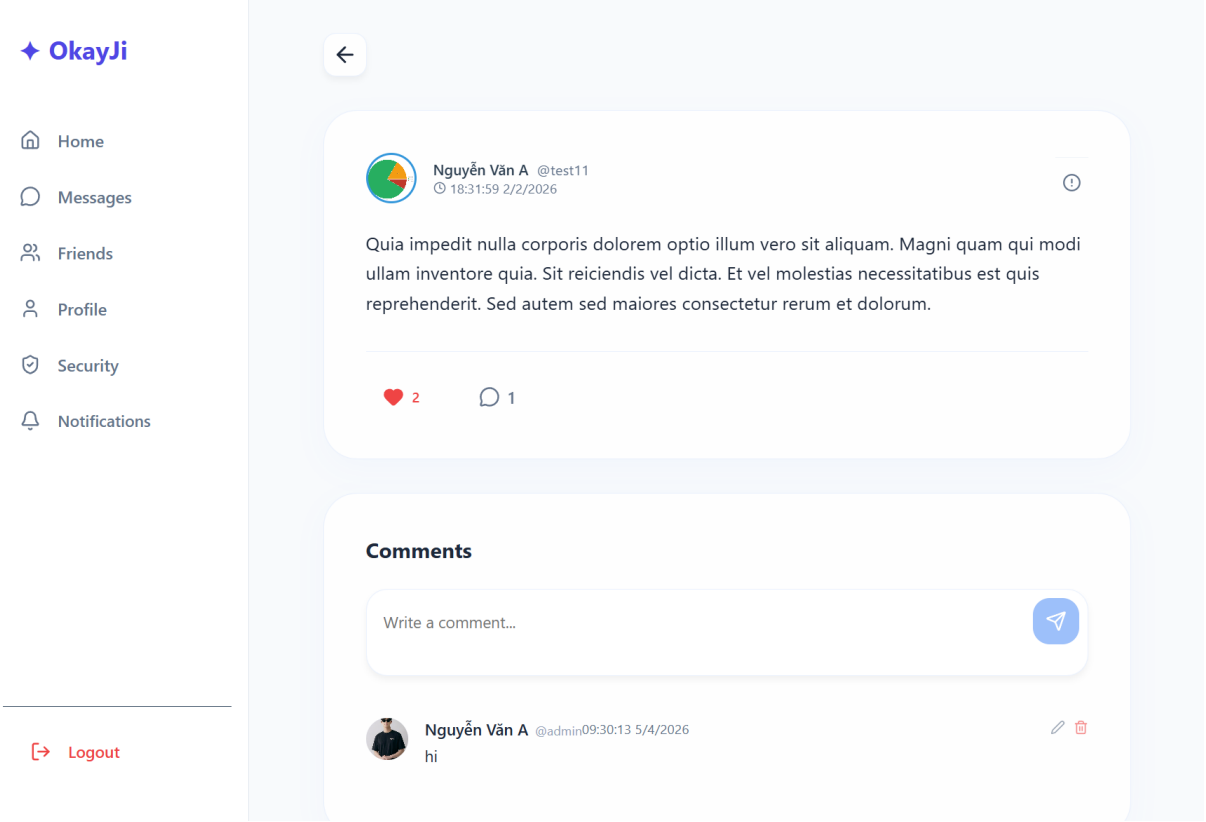
5.2. Giao diện bảng tin, bài viết và tương tác xã hội

Bảng tin là màn hình trung tâm của hệ thống, nơi hiển thị các bài viết của người dùng và bạn bè, đồng thời hỗ trợ phân trang để giảm tải dữ liệu và cải thiện tốc độ hiển thị. Tại đây, người dùng có thể tạo bài viết mới với nội dung văn bản, hình ảnh hoặc video.

Giao diện bảng tin:

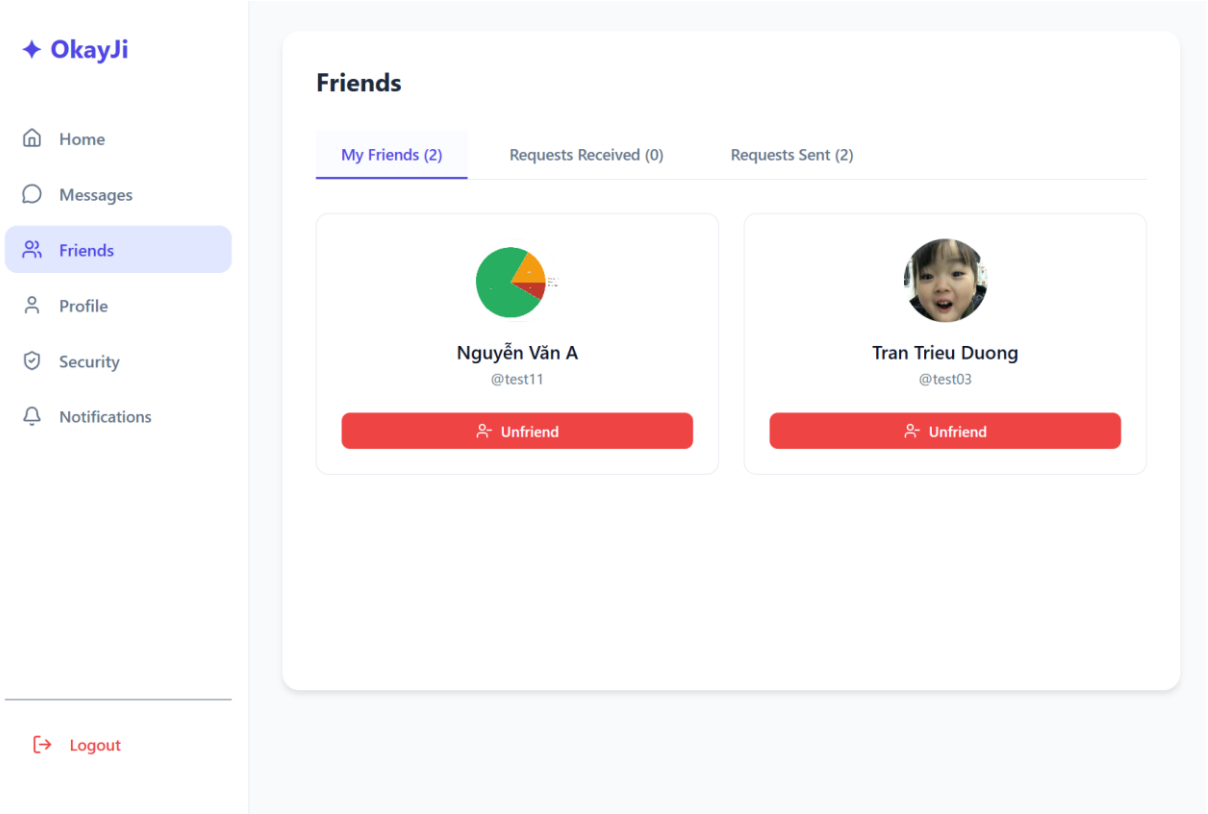


Giao diện chi tiết bài viết và bình luận:

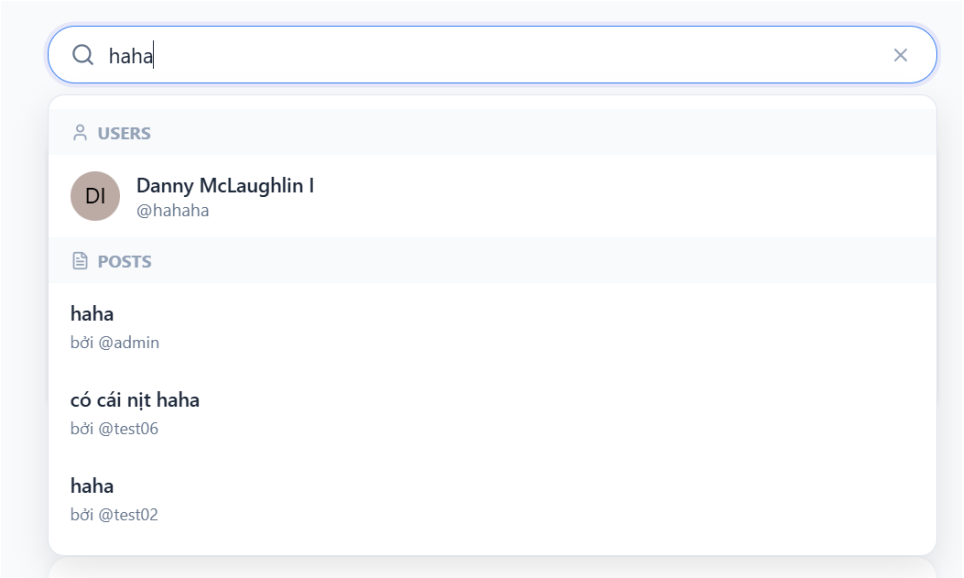


Người dùng có thể bày tỏ sự quan tâm thông qua chức năng thích bài viết và tham gia thảo luận bằng cách tạo, sửa hoặc xóa bình luận. Các chức năng này phản ánh đúng yêu cầu nghiệp vụ đã đặt ra cho phân hệ bài viết và tương tác trong hệ thống mạng xã hội OkayJi.

Giao diện danh sách bạn bè và lời mời kết bạn:



Giao diện tìm kiếm:

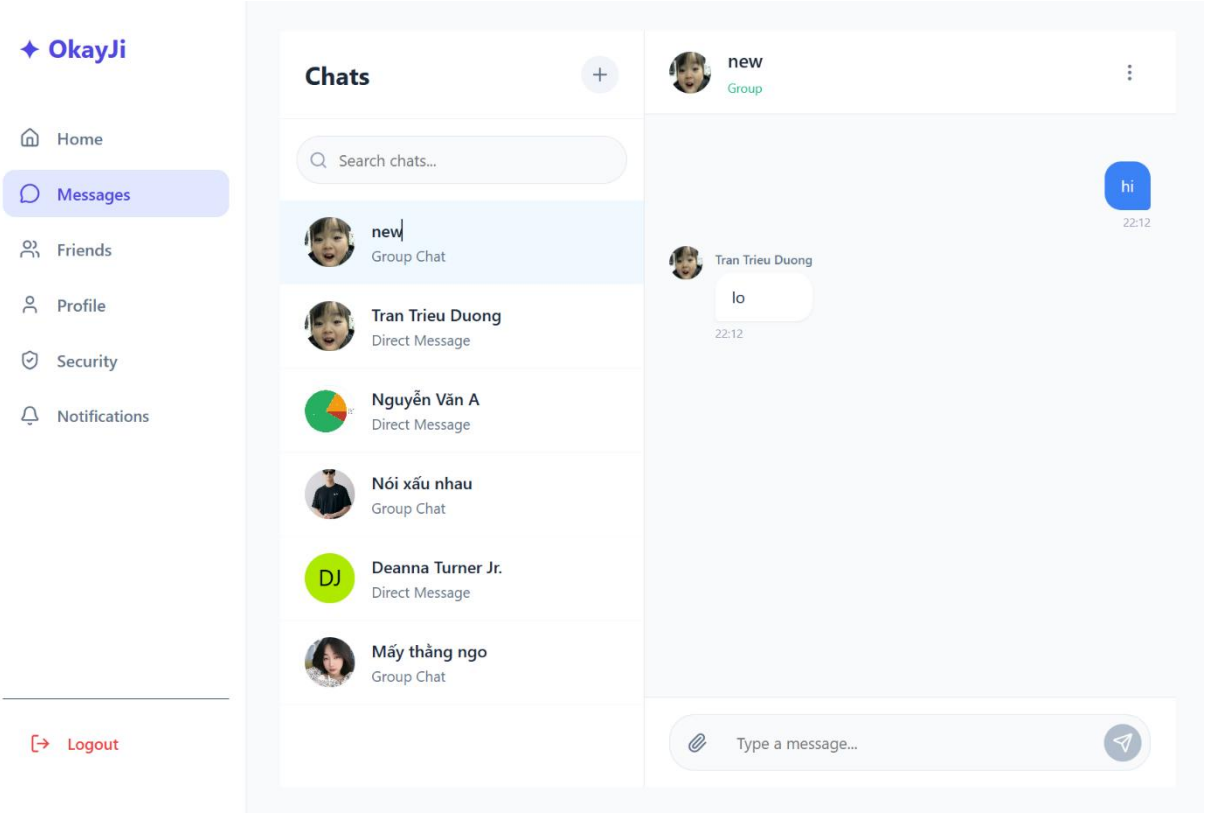


Hệ thống cũng hỗ trợ tìm kiếm người dùng và bài viết, giúp người dùng truy xuất nhanh nội dung quan tâm trong khi vẫn phải tuân thủ các giới hạn về quyền riêng tư đã được cấu hình. Đây là phần giao diện quan trọng nhằm tăng tính tiện dụng và cải thiện trải nghiệm sử dụng hằng ngày trên nền tảng.

5.3. Giao diện nhắn tin và thông báo

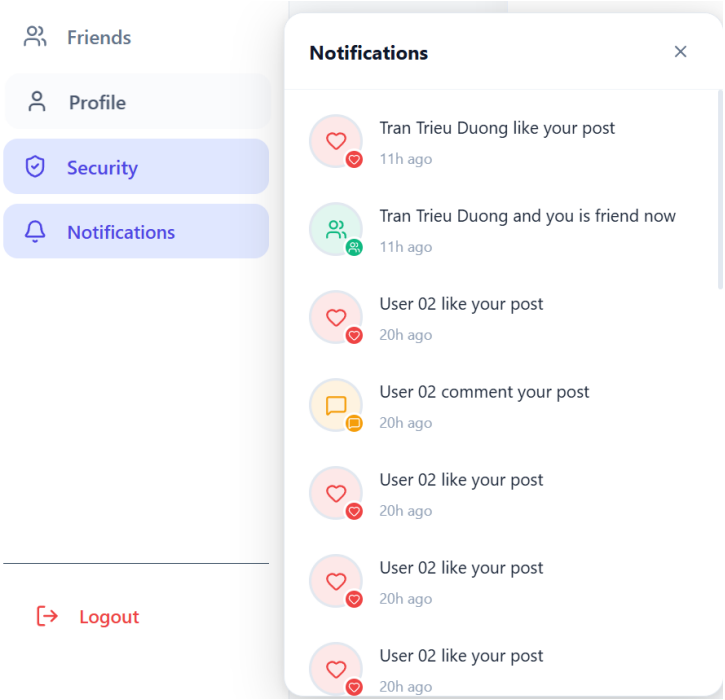
Một điểm nổi bật của OkayJi là hỗ trợ giao tiếp thời gian thực thông qua nhắn tin cá nhân và nhắn tin nhóm. Giao diện chat cho phép người dùng theo dõi danh sách hội thoại, gửi và nhận tin nhắn gần như ngay lập tức, đồng thời hỗ trợ nhiều loại nội dung như văn bản, hình ảnh và tệp đính kèm.

Giao diện nhắn tin thời gian thực:



Cơ chế chat của hệ thống được xây dựng dựa trên WebSocket với STOMP, phía front-end quản lý kết nối trong ChatContext và phía back-end tổ chức xử lý trong module chat kết hợp notification.

Giao diện thông báo:



Bên cạnh chat, giao diện thông báo giúp người dùng nhận được các sự kiện như có lượt thích, bình luận mới, lời mời kết bạn hoặc lời mời kết bạn được chấp nhận. Các thông báo được cập nhật theo thời gian thực, qua đó tăng khả năng tương tác và giúp người dùng không bỏ lỡ các hoạt động liên quan đến tài khoản của mình.

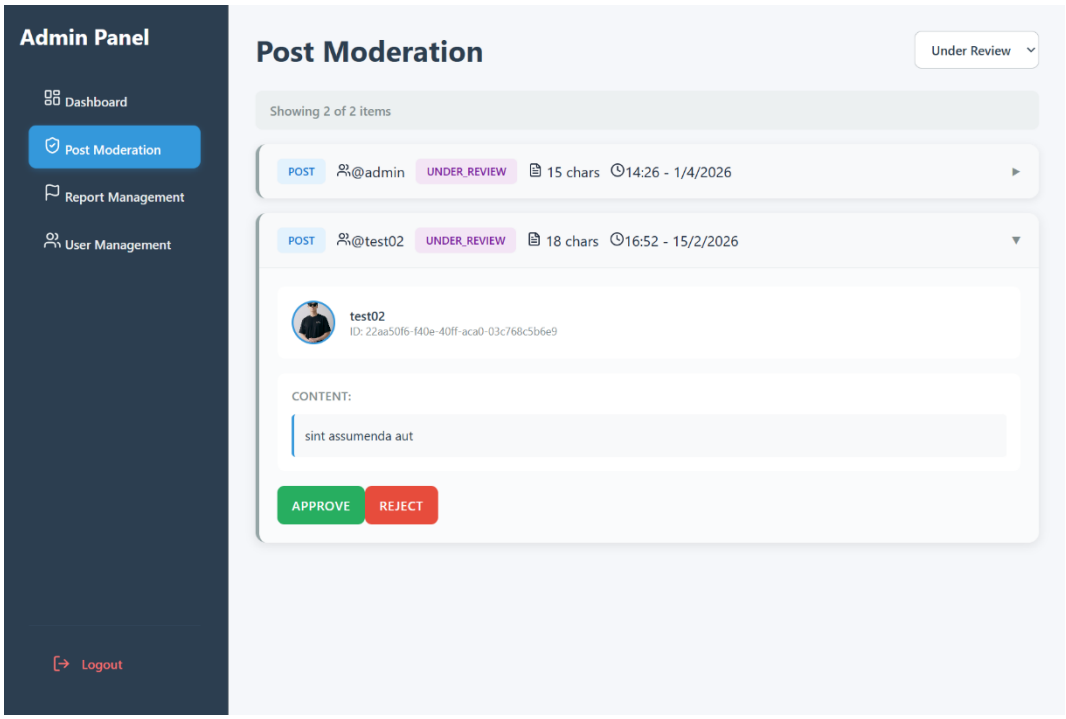
5.4. Giao diện Admin

Giao diện dashboard:

Trang Dashboard là màn hình tổng quan dành cho quản trị viên, giúp theo dõi nhanh tình trạng vận hành của hệ thống thông qua các thẻ thống kê như số lượng nội dung đã duyệt, số nội dung bị chặn, số nội dung đang xem xét, tổng số bài viết và tổng số người dùng. Ngoài ra, trang còn hiển thị biểu đồ thống kê số lượng người dùng mới theo từng tháng trong năm, hỗ trợ quản trị viên đánh giá mức độ tăng trưởng và hiệu quả hoạt động của nền tảng.

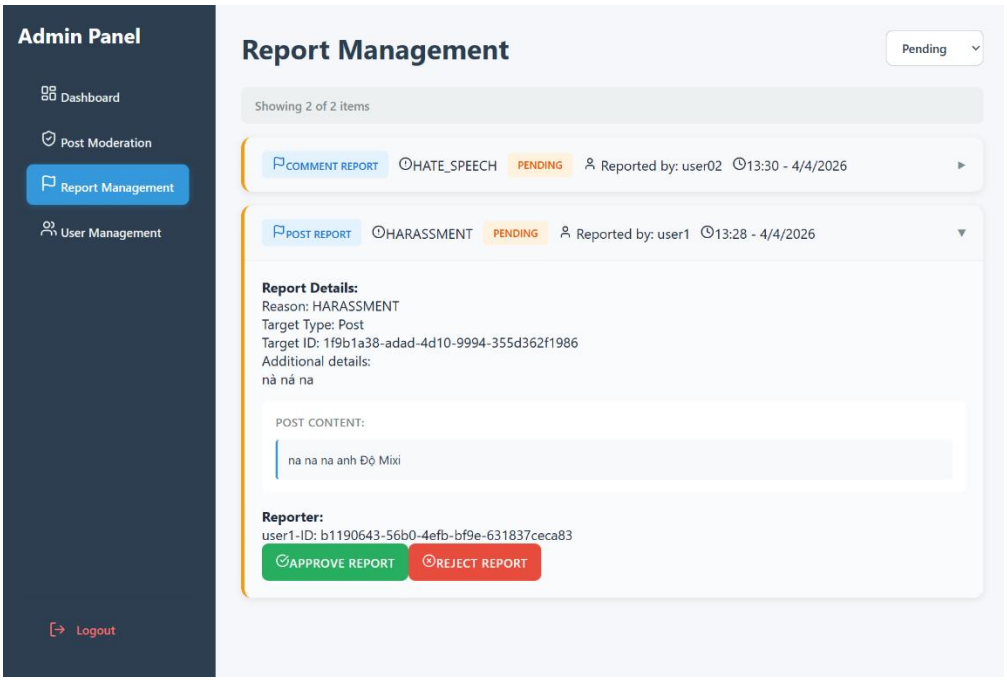
Trong giao diện này, thanh điều hướng bên trái giúp quản trị viên truy cập nhanh đến các phân hệ như Dashboard, Post Moderation, Report Management và User Management, đồng thời nút làm mới dữ liệu hỗ trợ cập nhật lại các số liệu thống kê khi cần thiết.

Giao diện kiểm duyệt bài viết:



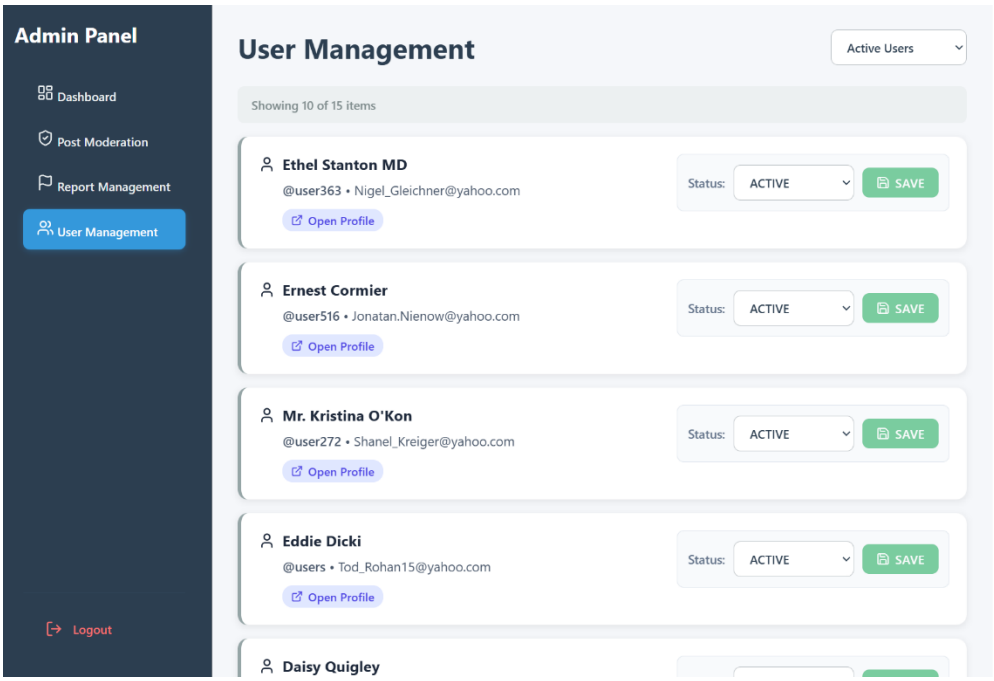
Trang Post Moderation phục vụ cho chức năng kiểm duyệt thủ công các bài đăng chưa được AI đánh giá chính xác hoặc đang ở trạng thái chờ xem xét. Giao diện cho phép quản trị viên lọc danh sách bài viết theo trạng thái, xem nhanh thông tin bài đăng như người đăng, thời gian tạo, độ dài nội dung và mở rộng từng mục để đọc chi tiết trước khi đưa ra quyết định xử lý.

Giao diện quản lý báo cáo vi phạm:



Trang Report Management được xây dựng để tiếp nhận và xử lý các báo cáo vi phạm do người dùng gửi lên đối với bài viết hoặc bình luận. Tại đây, quản trị viên có thể lọc báo cáo theo trạng thái xử lý, xem danh sách báo cáo hiện có, đồng thời mở rộng từng báo cáo để theo dõi đầy đủ các thông tin như loại báo cáo, lý do vi phạm, đối tượng bị báo cáo, nội dung chi tiết và thông tin người báo cáo.

Giao diện quản lý người dùng:



Trang User Management hỗ trợ quản trị viên theo dõi danh sách tài khoản trong hệ thống và thay đổi trạng thái hoạt động của từng người dùng. Giao diện hiển thị các thông tin cơ bản như tên người dùng, username, email, đồng thời cho phép lọc danh sách theo từng trạng thái và thao tác trực tiếp trên từng tài khoản.

5.5. Kiểm thử hệ thống

Nhóm tiến hành kiểm thử dựa trên các chức năng cốt lõi đã được xác định trong Chương 1 và các nhóm API được mô tả trong README của dự án. Quá trình kiểm thử tập trung vào tính đúng đắn của luồng nghiệp vụ, khả năng phản hồi giao diện, cơ chế phân quyền và hoạt động của các chức năng thời gian thực. Công cụ hỗ trợ kiểm thử bao gồm giao diện web phía front-end, các API phía back-end và tài liệu Swagger UI.

Các ca kiểm thử tiêu biểu được thực hiện như sau:

- **TC01 – Đăng ký và đăng nhập:** Người dùng tạo tài khoản mới, đăng nhập vào hệ thống và nhận token xác thực để sử dụng các chức năng yêu cầu quyền truy cập. Kết quả mong đợi là hệ thống tạo tài khoản hợp lệ, đăng nhập thành công và cho phép truy cập vào các màn hình chức năng sau khi xác thực. Kết quả ghi nhận: Đạt.
- **TC02 – Ghi nhớ đăng nhập:** Người dùng đăng nhập với tùy chọn ghi nhớ phiên làm việc để hệ thống lưu token trong thời gian dài hơn. Kết quả mong đợi là người dùng vẫn duy trì được trạng thái đăng nhập sau khi tải lại trang trong phạm vi thời gian đã cấu hình. Kết quả ghi nhận: Đạt.
- **TC03 – Cập nhật hồ sơ cá nhân:** Người dùng chỉnh sửa avatar, bio, giới tính, ngày sinh và quyền riêng tư hồ sơ. Kết quả mong đợi là hệ thống lưu đúng thông tin đã thay đổi và hiển thị lại dữ liệu mới trên giao diện hồ sơ. Kết quả ghi nhận: Đạt.
- **TC04 – Tạo, sửa và xóa bài viết:** Người dùng đăng bài viết mới có nội dung văn bản hoặc media, sau đó thực hiện chỉnh sửa và xóa bài viết của mình. Kết quả mong đợi là hệ thống xử lý đúng theo nghiệp vụ, cập nhật bài viết trên bảng tin và phản ánh đúng trạng thái của bài đăng. Kết quả ghi nhận: Đạt.
- **TC05 – Thích và bình luận bài viết:** Người dùng thực hiện lượt thích, tạo bình luận mới và chỉnh sửa hoặc xóa bình luận trên bài viết. Kết quả mong đợi là số

lượng tương tác được cập nhật đúng và nội dung bình luận được hiển thị tương ứng với thao tác của người dùng. Kết quả ghi nhận: Đạt.

- **TC06 – Kết bạn:** Người dùng gửi lời mời kết bạn, chấp nhận hoặc từ chối lời mời và thực hiện hủy kết bạn khi cần. Kết quả mong đợi là danh sách bạn bè và trạng thái lời mời được đồng bộ chính xác trên giao diện. Kết quả ghi nhận: Đạt.
- **TC07 – Tìm kiếm người dùng và bài viết:** Người dùng nhập từ khóa để tìm kiếm tài khoản hoặc nội dung quan tâm. Kết quả mong đợi là hệ thống trả về danh sách phù hợp với từ khóa tìm kiếm và vẫn tuân thủ các ràng buộc về quyền riêng tư. Kết quả ghi nhận: Đạt.
- **TC08 – Nhắn tin thời gian thực:** Hai người dùng gửi và nhận tin nhắn trong hội thoại trực tiếp hoặc trong nhóm chat. Kết quả mong đợi là tin nhắn xuất hiện gần như ngay lập tức ở phía người nhận, đồng thời lịch sử hội thoại được tải lại chính xác theo từng đợt. Kết quả ghi nhận: Đạt.
- **TC09 – Thông báo thời gian thực:** Hệ thống phát sinh thông báo khi có lượt thích, bình luận hoặc lời mời kết bạn. Kết quả mong đợi là người dùng nhận được thông báo kịp thời trên giao diện mà không cần tải lại toàn bộ trang. Kết quả ghi nhận: Đạt.
- **TC10 – Upload tệp:** Người dùng tải ảnh hoặc video lên hệ thống thông qua phân hệ quản lý tệp. Kết quả mong đợi là tệp được lưu trữ an toàn trên AWS S3, có thể truy cập bằng presigned URL và hỗ trợ tốt cho các tệp có dung lượng lớn thông qua multipart upload. Kết quả ghi nhận: Đạt.
- **TC11 – Kiểm duyệt nội dung tự động:** Khi người dùng đăng bài viết hoặc bình luận, hệ thống tạo ModerationJob và chuyển nội dung sang tiến trình đánh giá tự động bằng AI. Kết quả mong đợi là nội dung phù hợp được cho phép hiển thị, còn nội dung có dấu hiệu vi phạm sẽ bị từ chối hoặc chuyển sang trạng thái chờ xem xét. Kết quả ghi nhận: Đạt.
- **TC12 – Báo cáo vi phạm và kiểm duyệt thủ công:** Người dùng gửi báo cáo đối với bài viết hoặc bình luận vi phạm, sau đó quản trị viên tiếp nhận và xử lý trên giao diện quản trị. Kết quả mong đợi là quản trị viên có thể xem báo cáo, thay đổi

trạng thái nội dung và đưa ra quyết định chấp nhận hoặc bác bỏ báo cáo. Kết quả ghi nhận: Đạt.

- **TC13 – Thống kê vận hành hệ thống:** Quản trị viên truy cập khu vực quản trị để xem tổng số người dùng, tổng số bài đăng và số lượng người dùng mới theo năm. Kết quả mong đợi là hệ thống hiển thị được các số liệu tổng quan phục vụ đánh giá hoạt động của nền tảng. Kết quả ghi nhận: Đạt.

CHƯƠNG 6: KHÓ KHĂN VÀ HƯỚNG MỞ RỘNG

6.1. Khó khăn trong phân tích và thiết kế hệ thống

Khó khăn đầu tiên của nhóm là phải phân tích đầy đủ yêu cầu của một nền tảng mạng xã hội hiện đại, bởi hệ thống không chỉ dừng ở các chức năng cơ bản như đăng ký, đăng nhập hay đăng bài, mà còn bao gồm quyền riêng tư hồ sơ, quản lý quan hệ bạn bè, chat cá nhân và nhóm, thông báo real-time, báo cáo vi phạm, kiểm duyệt thủ công và thống kê quản trị. Việc bao quát đầy đủ các nghiệp vụ này đòi hỏi nhóm phải xác định rõ phạm vi chức năng ngay từ đầu để tránh thiếu sót hoặc chồng chéo giữa các phần của hệ thống.

Bên cạnh đó, mô hình dữ liệu của hệ thống tương đối lớn với nhiều thực thể như User, Profile, Role, Post, Comment, Reaction, Friend, FriendRequest, Chat, Message, ModerationJob, ModerationResult, Notification và Report. Việc xây dựng các mối quan hệ hợp lý giữa các thực thể này là một thách thức quan trọng, vì chỉ cần thiết kế thiếu chặt chẽ thì các chức năng như kết bạn, nhắn tin, kiểm duyệt nội dung hoặc thông báo sẽ dễ phát sinh lỗi và khó mở rộng về sau.

Ngoài yêu cầu đúng nghiệp vụ, nhóm còn phải bảo đảm hệ thống có khả năng bảo trì và mở rộng, nên kiến trúc phần mềm được định hướng theo mô hình phân tầng controller, service, repository và áp dụng các nguyên tắc SOLID. Điều này giúp hệ thống rõ ràng hơn về trách nhiệm của từng lớp, nhưng cũng làm tăng độ khó trong giai đoạn thiết kế ban đầu vì nhóm phải cân bằng giữa tiến độ hoàn thành đồ án và chất lượng kiến trúc tổng thể.

6.2. Khó khăn trong triển khai kỹ thuật

Một trong những khó khăn lớn nhất là triển khai các chức năng thời gian thực, đặc biệt là chat và thông báo, khi hệ thống phải kết hợp đồng thời REST API, WebSocket, STOMP và cơ chế xác thực JWT. So với các chức năng CRUD thông thường, các luồng giao tiếp hai chiều này phức tạp hơn vì cần đồng bộ dữ liệu liên tục giữa client và server, đồng thời vẫn phải bảo đảm tính đúng đắn của trạng thái phiên làm việc và quyền truy cập của người dùng.

Ở phía front-end, việc tổ chức state cho các màn hình feed, hồ sơ cá nhân, chat

và thông báo cũng tạo ra áp lực đáng kể trong quá trình phát triển. Hệ thống sử dụng kết hợp React Context API, local state và localStorage để quản lý dữ liệu toàn cục, dữ liệu cục bộ và token đăng nhập, nên nhóm phải xử lý cẩn thận để tránh tình trạng dữ liệu không đồng bộ hoặc giao diện cập nhật chậm khi có sự kiện thời gian thực.

Ngoài ra, đề tài còn tích hợp nhiều dịch vụ ngoài như AWS S3 để lưu trữ ảnh và video, OpenAI API để kiểm duyệt nội dung, và FFmpeg để xử lý video. Việc cấu hình môi trường, quản lý khóa bí mật, xử lý lỗi từ dịch vụ bên ngoài, hỗ trợ multipart upload và triển khai cơ chế retry cho moderation là những vấn đề kỹ thuật khiến quá trình cài đặt, kiểm thử và vận hành trở nên phức tạp hơn đáng kể.

6.3. Hướng mở rộng của hệ thống

Mặc dù còn một số khó khăn trong quá trình thực hiện, nền tảng hiện tại của OkayJi đã được tổ chức khá rõ ràng với mô hình client-server, back-end phân chia theo module nghiệp vụ và front-end xây dựng theo hướng pages, components, context. Đây là cơ sở phù hợp để tiếp tục mở rộng hệ thống trong tương lai mà không phải thay đổi toàn bộ cấu trúc đã xây dựng.

Nhóm đã xác định một số hướng phát triển tiếp theo như tích hợp OAuth2 Social Login với Google hoặc Facebook, bổ sung Redis Cache cho feed và dữ liệu người dùng, sử dụng Elasticsearch cho tìm kiếm full-text, ứng dụng Kafka hoặc message queue cho xử lý bất đồng bộ, triển khai Docker hoặc Docker Compose, xây dựng CI/CD Pipeline, phát triển tính năng Story hoặc Reel và gợi ý bạn bè bằng AI. Đây đều là những hướng mở rộng hợp lý vì chúng bám sát đặc thù của một nền tảng mạng xã hội và phù hợp với kiến trúc hiện tại của dự án.

Trong đó, OAuth2 Social Login sẽ giúp người dùng đăng nhập nhanh và thuận tiện hơn, Redis có thể hỗ trợ cải thiện hiệu năng truy xuất ở các khu vực thường xuyên đọc dữ liệu như bảng tin hoặc danh sách bạn bè, còn Elasticsearch sẽ nâng cao chất lượng tìm kiếm người dùng và bài viết. Bên cạnh đó, Kafka và các công cụ triển khai như Docker, CI/CD sẽ giúp hệ thống tiến gần hơn đến mô hình vận hành thực tế, đặc biệt khi quy mô người dùng tăng và cần nhiều xử lý bất đồng bộ hơn.

Ngoài các hướng đã được nêu, hệ thống cũng có thể tiếp tục mở rộng ở phần quản

trị và trải nghiệm người dùng, chẳng hạn như tăng cường thống kê vận hành, hoàn thiện cơ chế kiểm duyệt nội dung đa phương tiện, hoặc phát triển thêm các tính năng cá nhân hóa bảng tin và gợi ý nội dung. Những định hướng này phù hợp với mục tiêu ban đầu của đề tài là xây dựng một hệ thống mạng xã hội hiện đại, an toàn, có khả năng mở rộng và dễ bảo trì trong thời gian dài.

PHẦN KẾT LUẬN

Sau quá trình nghiên cứu, phân tích, thiết kế và xây dựng, đề tài "Xây dựng website mạng xã hội OkayJi" đã hoàn thành mục tiêu phát triển một hệ thống mạng xã hội hoạt động theo mô hình client-server với các chức năng cốt lõi phục vụ kết nối, chia sẻ và tương tác giữa người dùng. Hệ thống đã hiện thực các chức năng quan trọng như đăng ký, đăng nhập, quản lý hồ sơ cá nhân, đăng bài viết, thích và bình luận, kết bạn, nhắn tin cá nhân hoặc theo nhóm, thông báo thời gian thực, quản lý tệp, báo cáo vi phạm và các chức năng quản trị dành cho quản trị viên.

Về mặt kỹ thuật, dự án đã kết hợp front-end sử dụng ReactJS và Vite với back-end sử dụng Java Spring Boot, đồng thời ứng dụng MySQL cho lưu trữ dữ liệu, JWT và Spring Security cho xác thực và phân quyền, WebSocket cho các chức năng thời gian thực, AWS S3 cho lưu trữ media và OpenAI API cho kiểm duyệt nội dung tự động. Việc tổ chức hệ thống theo các lớp controller, service, repository và theo các module nghiệp vụ riêng biệt đã giúp mã nguồn rõ ràng hơn, thuận lợi cho bảo trì và mở rộng trong tương lai. Bên cạnh đó, việc áp dụng các nguyên tắc SOLID trong quá trình thiết kế cũng góp phần nâng cao chất lượng kiến trúc phần mềm và giảm sự phụ thuộc chéo giữa các thành phần trong hệ thống.

Thông qua quá trình thực hiện đề tài, nhóm không chỉ củng cố được kiến thức về phát triển ứng dụng web hiện đại mà còn rèn luyện thêm kỹ năng phân tích yêu cầu, thiết kế hệ thống, làm việc nhóm, tổ chức mã nguồn, kiểm thử và tích hợp nhiều công nghệ khác nhau trong cùng một sản phẩm phần mềm. Mặc dù hệ thống vẫn còn khả năng tiếp tục hoàn thiện và mở rộng thêm nhiều tính năng trong tương lai, kết quả hiện tại đã cho thấy OkayJi là một nền tảng có định hướng phát triển rõ ràng, đáp ứng tương đối đầy đủ các yêu cầu đã đặt ra ban đầu của đề tài.

Từ kết quả đạt được, đề tài có thể xem là một nền tảng thực nghiệm có giá trị trong học tập và nghiên cứu, đồng thời là cơ sở để tiếp tục phát triển thành một hệ thống hoàn chỉnh hơn với khả năng mở rộng về hiệu năng, chức năng và trải nghiệm người dùng trong các giai đoạn tiếp theo.

TÀI LIỆU THAM KHẢO

[1] Spring Security with JWT, Bootify.io, https://bootify.io/spring-security/rest-api-spring-security-with-jwt.html

[2] Using WebSocket to build an interactive web application, Spring.io, https://spring.io/guides/gs/messaging-stomp-websocket

[3] Build a React app from Scratch, React.dev, https://react.dev/learn/build-a-react-app-from-scratch

[4] Work with Amazon S3 pre-signed URLs, AWS SDK for Java, https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/examples-s3-presign.html

[5] Moderation, OpenAI API docs, https://platform.openai.com/docs/guides/moderation

[6] SOLID Design Principles Explained, DigitalOcean, https://www.digitalocean.com/community/conceptual-articles/s-o-l-i-d-the-first-five-principles-of-object-oriented-design